



Principy programovacích jazyků a objektově orientovaného programování IPP – II Studijní opora

Zbyněk Křivka, Dušan Kolář
Ústav informačních systémů
Fakulta informačních technologií
VUT v Brně

Květen '03 – Únor '08, Červen '17, Březen '19
Verze 1.3b

Tento učební text vznikl za podpory projektu „Zvýšení konkurenceschopnosti IT odborníků – absolventů pro Evropský trh práce“, reg.č. CZ.04.1.03/3.2.15.1/0003. Tento projekt je spolufinancován Evropským sociálním fondem a státním rozpočtem České republiky.

Abstrakt

Objekty jsou všudypřítomnými stavebními bloky jak reálného, tak virtuálního světa. Zaměření se na objektově orientovanou tvorbu systémů však dosáhlo masivního rozšíření až v nedávných 90. letech minulého století. I přesto se stále jedná o velmi moderní principy využívané nejen při tvorbě softwarových produktů, ale i libovolných jiných systémů. Proto má smysl se tomuto stylu myšlení naučit a pochopit jeho základní koncepty i pokročilé vlastnosti.

Tato publikace je zaměřena především na popis programovacích jazyků, které tvoří komunikační most mezi člověkem a výpočetní technikou, tudíž i na objektovou orientaci budeme nahlížet z pohledu systémových analytiků a programátorů, jejichž základními nástroji jsou v této oblasti objektově orientované jazyky, prostředí a další nástroje podporující objektově orientovaný přístup k analýze, návrhu a implementaci požadovaného systému.

Věnováno Mirkovi a Romanovi

Vřelé díky všem, kdo nás podporovali a povzbuzovali při práci na této publikaci.

Obsah

1	Úvod	5
1.1	Koncepce modulu	8
1.2	Potřebné vybavení	9
2	Principy objektově orientovaných jazyků	11
2.1	Základní charakteristika	13
2.1.1	Historie	14
2.1.2	Základní pojmy	14
2.1.3	Základní koncepty OOP	15
2.1.4	Minimální model výpočtu	18
2.1.5	Výhody a nevýhody OOP	19
2.2	Datové a řídicí abstrakce	20
2.2.1	Třídně orientované jazyky	21
2.2.2	Prototypově orientované jazyky	31
2.3	Závěr	35
2.4	Studijní literatura	36
3	Formalismy a jejich užití	37
3.1	Formální základ popisu objektově orientovaného jazyka	39
3.2	ς -kalkul	39
3.2.1	Syntaxe a sémantika ς -kalkulu	40
3.2.2	Příklady	42
3.3	UML - formální vizuální jazyk	44
3.3.1	Výhody a nevýhody formálního návrhu	45
3.4	Závěr	45
4	UML	47
4.1	Co je to UML?	49
4.2	Modelování v UML	49
4.2.1	Stavební bloky	50
4.2.2	Diagramy tříd a objektů	51
4.2.3	Další diagramy UML	56
4.3	Závěr	60

5	Vlastnosti objektově orientovaných jazyků	61
5.1	Úvod	63
5.2	Poznámka ke klasifikaci jazyků	63
5.2.1	Pojmy	63
5.2.2	Klasifikace jazyků	64
5.3	Vlastnosti třídních jazyků	65
5.3.1	Řízení toku programu	66
5.3.2	Jmenné prostory	66
5.3.3	Modifikátory viditelnosti	66
5.3.4	Přetěžování metod	67
5.3.5	Vícenásobná dědičnost	67
5.3.6	Rozhraní	69
5.3.7	Výjimky	71
5.3.8	Šablony	72
5.3.9	Systémy s rolemi	74
5.4	Poznámky k implementaci OOJ	76
5.4.1	Manipulace se třídami	76
5.4.2	Virtuální stroj	78
5.4.3	Poznámka o návrhových vzorech	80
5.5	Zpracování - analýza, vyhodnocení, interpretace, překlad	81
5.5.1	Překladač	81
5.5.2	Interpret	82
5.6	Závěr	83
6	Závěr	87
	Rejstřík	89
	Literatura	95

Notace a konvence použité v publikaci

Každá kapitola a celá tato publikace je uvozena informací o čase, který je potřebný ke zvládnutí dané oblasti. Čas uvedený v takovéto informaci je založen na zkušenostech více odborníků z oblasti a uvažuje čas nutný k pochopení prezentovaného tématu. Tento čas nezahrnuje dobu nutnou pro opakované memorování paměťově náročných statí, neboť tato schopnost je u člověka silně individuální. Příklad takového časového údaje následuje.

Čas potřebný ke studiu: 2 hodiny 15 minut

Podobně jako dobu strávenou studiem můžeme na začátku každé kapitoly či celé publikace nalézt cíle, které si daná pasáž klade za cíl vysvětlit, kam by mělo studium směřovat a čeho by měl na konci studia dané pasáže studující dosáhnout, jak znalostně, tak dovednostně. Cíle budou v kapitole vypadat takto:

Cíle kapitoly

Cíle kapitoly budou poměrně krátké a stručné, v podstatě shrnující obsah kapitoly do několika málo vět či odrážek.

Poslední, nicméně stejně důležitý údaj, který najdeme na začátku kapitoly, je průvodce studiem. Jeho posláním je poskytnout jakýsi návod, jak postupovat při studiu dané kapitoly, jak pracovat s dalšími zdroji, v jakém sledu budou jednotlivé cíle kapitoly vysvětleny apod. Notace průvodce je taktéž standardní:

Průvodce studiem

Průvodce je často delší než cíle, je více návodný a jde jak do šířky, tak do hloubky, přitom ho nelze považovat za rozšíření cílů, či jakýsi abstrakt dané statí.

Za průvodcem bude vždy uveden obsah kapitoly.

Následující typy zvýrazněných informací se nacházejí uvnitř kapitol či podkapitol, a i když se zpravidla budou vyskytovat v každé kapitole, jejich výskyt a pořadí není nijak pevně definováno. Uvedení logické oblasti, kterou by bylo vhodné studovat naráz je označeno slovem „Výklad“ takto:

Výklad

Důležité nebo nové pojmy budou definovány a tyto definice budou číslovány. Důvodem je možnost odkazovat již jednou definované pojmy, a tak významně zeshlíhet a zpřehlednit text v této publikaci. Příklad definice je uveden vzápětí:

Definice!

Definice 0.0.1 Každá definice bude využívat poznámku na okraji k tomu, aby upozornila na svou existenci. Jinak je možné zvětšený okraj použít pro vpisování vlastních poznámek. První číslo v číselné identifikaci definice (či algoritmu, viz níže) je číslo kapitoly, kde se nacházela, druhé je číslo podkapitoly a třetí je pořadí samotné entity v rámci podkapitoly.

Pokud se bude někde vyskytovat určitý postup či konkrétní algoritmus, bude také označen, podobně jako definice. I číslování bude mít stejný charakter a logiku.

Algoritmus!

Algoritmus 0.0.1 Pokud je čtenář zdatný v oblasti, kterou kapitola či úsek výkladu prezentuje, potom je možné přejít na další oddíl stejné úrovně.

Přeskoky v rámci jednoho oddílu však nedoporučujeme.

V průběhu výkladu se navíc budou vyskytovat tzv. řešené příklady. Jejich zadání bude jako jakékoliv jiné, ale kromě něj budou obsahovat i řešení s nástinem postupu, jak je takové řešení možné získat. V případě, že by řešení vyžadovalo neúměrnou část prostoru, bude vhodným způsobem zkráceno tak, aby podstata řešení zůstala zachována.

Řešený příklad

Zadání: Vyjmenujte typy rozlišovaných textů, které byly doposud v textu zmíněny.

Řešení: Doposud byly zmíněny tyto rozlišené texty:

- Čas potřebný ke studiu
- Cíle kapitoly
- Průvodce studiem
- Definice
- Algoritmus
- Právě zmiňovaný je potom Řešený příklad

Některé informace mohou být vypíchnuty či doplněny takto bokem. V závěru každého výkladového oddílu se potom bude možné setkat s opětovným zvýrazněním důležitých pojmů, které se v dané části vyskytly, a případně s úlohou, která slouží pro samostatné prověření schopností a dovedností, které daná část vysvětlovala.

Pojmy k zapamatování

- Rozlišené texty
- Mezi rozlišené texty patří: čas potřebný ke studiu, cíle kapitoly, průvodce studiem, definice, algoritmus, řešený příklad.

Úlohy k procvičení:

Který typ rozlišeného textu se vyskytuje typicky v úvodu kapitoly.
Který typ rozlišeného textu se vyskytuje v závěru výkladové části?

Na konci každé kapitoly bude určité shrnutí obsahu a krátké resumé.

Závěr

V této úvodní stati publikace byly uvedeny konvence pro zvýraznění rozlišených textů. Zvýraznění textů a pochopení vazeb a umístění zvyšuje rychlost a efektivnost orientace v textu.

Pokud úlohy určené k samostatnému řešení budou vyžadovat nějaký zvláštní postup, který nemusí být okamžitě zřejmý, což lze odhalit tím, že si řešení úlohy vyžaduje enormní množství času, tak je možné nahlédnout k nápovědě, která říká jak, případně kde nalézt podobné řešení, nebo další informace vedoucí k jeho řešení.

Klíč k řešení úloh

Rozlišený text se odlišuje od textu běžného změnou podbarvení či ohrazením.

Možnosti dalšího studia či možnosti, jak dále rozvíjet danou tematiku, jsou shrnuty v poslední nepovinné části kapitoly, která odkazuje, ať přesně či obecně, na další možné zdroje zabývající se danou problematikou.

Další zdroje

Oblasti, které studují formát textu určeného pro distanční vzdělávání a samostudium, se pojí se samotným termínem distančního či kombinovaného studia (distant learning) či tzv. e-learningu.

Kapitola 1

Úvod

Čas potřebný ke studiu: 16 hodin a 30 minut

Tento čas reprezentuje dobu pro studium celého modulu.

Údaj je pochopitelně silně individuální záležitostí a závisí na současných znalostech a schopnostech studujícího. Proto je vhodné jej brát v úvahu pouze orientačně a po nastudování prvních kapitol si provést vlastní korekci.

Cíle kapitoly

Cílem modulu je seznámit studujícího s širokou a moderní oblastí objektově orientovaných programovacích jazyků (dále OOJ), které byly zmíněny již v prvním modulu (IPP I) v rámci kapitoly Klasifikace jazyků. Ty modul studuje z hlediska nových přístupů k programování a myšlení nejen při samotné implementaci objektových systémů, ale i při jejich analýze a návrhu. Neopomíná samozřejmě základní vlastnosti takovýchto jazyků spolu s naznačenými možnostmi využití, implementace i modelování.

Po ukončení studia modulu:

- budete chápat základní paradigmatu objektové orientace;
- budete schopni klasifikovat objektově orientované jazyky (OOJ) a jejich vlastnosti;
- budete schopni se na základě dosažených přehledových znalostí rozhodovat o kvalitě a použitelnosti konkrétních OOJ pro řešení konkrétních problémů;
- budete mít pasivní znalosti formálních aparátů pro popis syntaxe a sémantiky OOJ

- budete mít přehledové znalosti o modelování grafickým jazykem UML, které tak můžete samostatně dále rozvíjet.

Obsah

1.1	Koncepce modulu	8
1.2	Potřebné vybavení	9

Průvodce studiem

Modul začíná popisem paradigmat objektové orientace spolu se základní klasifikací OOJ. Dále potom postupuje podle jednoho zvoleného kritéria a předkládá vlastnosti OOJ z hlediska uživatelského i implementačního. Pozastavuje se nad možnými formalismy spojenými s takovými jazyky, včetně způsobu popisu objektových modelů.

Rozčlenění kapitol se pokoušelo udržet jejich vzájemnou relativně nízkou závislost. Proto je možné, pokud je vám nějaká oblast blízká, přejít vpřed na další oblast.

V první kapitole se probírají základní principy objektové orientace a základní pojmy z této oblasti, které student uplatní při studium libovolného konkrétního OOJ. Následuje krátká kapitola o formálním popisu OOJ a přehledová kapitola o modelovacím jazyce UML. Modul je zakončen pokročilou kapitolou o vlastnostech objektově orientovaných jazyků.

Pro studium modulu je důležité být aktivním programátorem a nově nabyté znalosti si prakticky odzkoušet. Aktivní užití některého z typických představitelů objektově orientovaných jazyků se tak stává jasnou výhodou jak při samotném studiu, tak při chápání širších souvislostí. Nestačí pochopit jen to, jak jazyk vypadá zvenčí (syntaxe a sémantika), ale i co se skrývá za tím, že se chová tak, jak se chová.

Návaznost na předchozí znalosti

Pro studium tohoto modulu je tedy nezbytné, aby měl studující základní znalosti ze strukturovaných a modulárních programovacích jazyků a, jak již bylo zmíněno, byl aktivním programátorem alespoň v jednom vyšším, nejlépe objektovém programovacím jazyce. Pojmem aktivní programátor se zde chápe člověk, který si bude prakticky ověřovat a zkoušet získané znalosti i úlohy napomáhající k pochopení probírané látky.

1.1 Koncepce modulu

Začíná se kapitolou nejobecnější, která je nutnou prerekvizitou pro studium každého objektově orientovaného jazyka nebo modelu. Následuje matematictější kapitola, která má za úkol nastínit formální aspekty práce s takovými jazyky, ale není zcela nezbytná pro pochopení zbývajících textů. Další část se věnuje především modelování a popisu analýzy a návrhu aplikací pomocí objektové orientace a grafického jazyka UML. Jedná se spíše o přehledovou kapitolu, která opakuje a případně rozšiřuje znalosti o UML z předchozího studia. Pro tuto oblast nebylo dostatek prostoru na vyčerpávající výklad, a to ani nemuselo — z důvodu dostupnosti velkého množství kvalitní literatury

na toto téma i v českém jazyce. Poslední pasáže modulu se věnují náročnějším vlastnostem OOJ a možnostem jejich implementace. Sice se nedovíte, jak přesně postupovat například při tvorbě překladače nebo celého prostředí nového OOJ, ale přesto se zde nalézají velmi hodnotné informace pro pochopení některých klíčových vlastností a omezení OOJ.

Studium modulu vyžaduje sekvenční přístup především v rámci jednotlivých kapitol.

1.2 Potřebné vybavení

Pro studium a úspěšné zvládnutí tohoto modulu není třeba žádné speciální vybavení. Je však *vhodné* doplnit text osobní zkušeností s nějakým reprezentantem objektově orientovaných jazyků (například Smalltalk, Java, C#, C++, příp. SELF). Potom je ovšem nutné mít přístup k odpovídající výpočetní technice a ke zdrojům nabízejícím překladače či interprety daných jazyků, což je v současnosti typicky Internet.

Další zdroje

Objektově orientované programovací jazyky jsou jednotlivě představovány v řadě publikací.

Zatímco u jazyků jako takových najdeme řadu i českých titulů (např. Eckel: Myslíme v jazyku C++, či Herout: Učebnice jazyka Java, nebo Bloch: Java efektivně), tak u obecnější teorie objektově orientovaných jazyků jsou tituly typicky anglické (např. Abadi, Cardelli: A Theory of Objects). Proto jsou v textu u klíčových termínů uváděny i odpovídající termíny anglické, které by měly usnadnit vyhledání relevantních odkazů na Internetu (např. www.google.com, www.wikipedia.org), který je v tomto směru bohatou studnicí znalostí, jež mohou vhodně doplnit a rozšířit studovanou problematiku. Je však nutné být při akceptování poznatků z Internetu obezřetný, protože se často nejedná o recenzované nebo jinak editorsky schválené texty, a ty mohou proto obsahovat i nepravdivé či nepřesné údaje.

Kapitola 2

Principy objektově orientovaných jazyků

Tato kapitola obsahuje popis základních vlastností a problémů objektově orientovaných jazyků (dále OOJ), které navíc již mají některé vlastnosti modulárních jazyků.

Čas potřebný ke studiu: 3 hodiny 15 minut.

Cíle kapitoly

Hlavní cílem kapitoly je seznámit čtenáře s hlavními koncepty objektově orientovaného paradigmatu a základními pojmy objektově orientovaných jazyků (OOJ), včetně jejich klasifikace. Především v druhé části kapitoly bude přehledovým způsobem vysvětlena řada nejčastěji používaných pojmů a definic v kontextu objektově orientovaných jazyků.

Průvodce studiem

Při studiu základních a později i pokročilých vlastností objektově orientovaných jazyků budou všechny pojmy vysvětleny za předpokladu použití čistě objektově orientovaného jazyka.

Konkrétní OOJ se pravděpodobně bude od ideálního stavu odchylovat a lišit, takže při následném studiu konkrétního OOJ (např. za asistence kvalitní odborné literatury) budete na tyto odlišnosti jistě upozorněni. Na některé nejznámější úskalí některých nejrozšířenějších OOJ krátce upozorníme již v tomto textu.

Obsah

2.1	Základní charakteristika	13
2.1.1	Historie	14
2.1.2	Základní pojmy	14
2.1.3	Základní koncepty OOP	15
2.1.4	Minimální model výpočtu	18
2.1.5	Výhody a nevýhody OOP	19
2.2	Datové a řídicí abstrakce	20
2.2.1	Třídně orientované jazyky	21
2.2.2	Prototypově orientované jazyky	31
2.3	Závěr	35
2.4	Studijní literatura	36

Výklad

2.1 Základní charakteristika

Tato skupina jazyků rozšiřuje modulární jazyky o možnost spojit konkrétní data i s operacemi, které je manipulují. Data stojí v centru pozornosti jak návrhářů jazyka, tak potom těch, kteří implementují program.

Zevrubně řečeno, objektově orientované programování využívá při psaní programů také dekompozici do modulů¹. Tyto moduly ovšem tvoří zcela samostatné celky a objektově orientované paradigma pevně určuje způsob práce a komunikace s těmito celky.

Definice 2.1.1 *Objektově orientované programování (OOP) je způsob abstrakce, kdy algoritmus implementujeme pomocí množiny zapouzdřených vzájemně komunikujících entit (objektů), z nichž každá má plnou výpočetní sílu celého počítače.* **Definice!**

Alan Kay

Objektově orientovaný přístup k programování je založen na intuitivní korespondenci mezi softwarovou simulací reálného systému a reálným systémem samotným. Analogie je především mezi vytvářením algoritmického modelu skutečného systému ze softwarových komponent a výstavbou mechanického modelu pomocí skutečných objektů. Podle této analogie i ony softwarové komponenty nazýváme *objekty*. Objektově orientované programování pak zahrnuje analýzu, návrh a implementaci aspektů, kde jsou reálné objekty nahrazeny těmi softwarovými (virtuálními).

Definice 2.1.2 *Objektově orientovaný systém (program, aplikace) se skládá z jednoho či více objektů, které spolu komunikují a interagují při spolupráci na řešení daného problému.* **Definice!**

Hlavní výhody objektové orientace bychom mohli shrnout do tří nejpodstatnějších bodů:

- analogie mezi softwarovým modelem a reálným modelem,
- flexibilita takovýchto softwarových modelů,
- a jejich znovupoužitelnost.

¹Obecně může být modularita objektově orientovaného jazyka na dvou úrovních: (1) objekt/třída jako modul a (2) množina souvisejících objektů/tříd jako modul, který lze samostatně kompilovat.

2.1.1 Historie

V roce 1966 vytvořili O.-J. Dahl a K. Nygaard simulační jazyk SIMULA, který je považován za první objektově orientovaný jazyk, který mimo jiné zavedl i pojem třída. O několik let později se v Palo Alto Research Center firmy Xerox podíleli dr. Alan Kay a Adele Goldbergová na vývoji prvního objektově orientovaného grafického systému, který si kladl za cíl sloučit prostředky pro ovládání systému a vývojové prostředí do jednoho jazyka, který byl nazván Smalltalk-72 a postupem času se vyvinul do dnešní standardní podoby Smalltalku-80. Tento projekt se zabýval vytvořením osobního počítače s grafickým uživatelským rozhraním. Světlo světa spatřily koncepty jako zasílání zpráv, dědičnost a grafické uživatelské rozhraní. V samotném jazyce Smalltalk je hodně stop po inspiraci prvním neimperativním jazykem LISP.

Idea objektově orientovaného programování se začala rychle rozrůstat v 70. a počátkem 80. let, kdy Bjorn Stroustrup integroval objektově orientované paradigma do jazyka C, a dal tak vzniknout v roce 1991 velmi populárnímu C++, který se stal prvním masově rozšířeným objektově orientovaným jazykem. Poté na začátku 90. let začala skupina společnosti Sun pod vedením Jamese Goslinga vyvíjet zjednodušenou verzi C++, kterou nazvali Java. Původně se mělo jednat o programovací jazyk pro video-on-demand aplikace, ale nakonec se projekt přeorientoval na internetové aplikace. Celosvětového rozšíření se tento jazyk dočkal relativně brzy po uvedení na trh, hlavně díky obrovské expanzi Internetu a dobrému marketingu.

Pro vývoj v OOJ je také z pohledu softwarového inženýrství velmi důležitá racionalizace zápisu s využitím grafického jazyka UML, který byl vypracováván od 90. let, především trojicí metodologů Grady Booch, Ivar Jacobson a Jim Rumbaugh, a hojně je využíván při objektově orientované analýze a návrhu dodnes (již verze 2.1).

2.1.2 Základní pojmy

objekt

Pojem *objekt* lze definovat hned několika způsoby a většinou záleží na úhlu pohledu. Žádná z definic se však naštěstí nevylučuje, a tak nic nebrání, abychom si jich uvedli více. Nejobecnější programátorský pohled definuje objekt jako jednoznačně identifikovatelný reálný objekt (se sémantikou z obecné češtiny) nebo abstrakci (to v případě, že reálný objekt popisujeme nějakým abstraktním modelem), která zahrnuje data a jejich chování (operace nad těmito daty). V případě jazyka založeného na třídách lze pojem objekt ještě přesněji popsat jako instanci třídy obsahující data a operace. Tento populárnější popis

lze ještě zobecnit a říci, že objekt je entita, která rozumí zaslání některých zpráv a ve své vnitřní struktuře umožňuje zapouzdřit další objekty (může se skládat z dalších objektů).

Definice 2.1.3 *Objekt je entita zapouzdřující stavové informace (data reprezentovaná dalšími objekty) a poskytující sadu operací nad tímto objektem nebo jeho částmi.* **Definice!**

Vnitřním buňkám obsahujícím tyto zapouzdřené další objekty (nebo reference na ně) budeme říkat *instanční proměnné* nebo také *atributy*² a budou tvořit pojmenované datové části objektu. Objekty vzájemně interagují a komunikují pomocí tzv. mechanismu *zasílání zpráv*. *Zpráva* je komunikační jednotka mezi dvěma libovolnými objekty. Kromě svého jména (tzv. *selektor*) typicky obsahuje odkaz na *příjemce* (objekt, kterému je zpráva zaslána) zprávy a dodatečné informace v podobě *parametrů* (argumentů), které slouží pro podrobnější specifikaci zprávy a doplnění informací předávaných mezi těmito objekty. Z hlediska modelu výpočtu je důležitý nejen příjemce, ale i *odesílatel* zprávy (objekt, v jehož kontextu byla prováděna metoda, ze které byla zpráva odeslána). Odesílatel však není součástí zprávy samotné, ale pomocných struktur pro objektový model výpočtu (typicky zásobník volání resp. zásobník kontextů). Její sémantika je pak taková, že příjemce (adresát) na obdrženou zprávu od odesílatele reaguje vyhledáním patřičné implementace reakce na tuto zprávu, což bývá nejčastěji odpovídající zapouzdřená funkce, kterou budeme nazývat *metoda*. Metody implementují veškeré chování objektů, nebo chcete-li reakce na obdržené zprávy, a mívají spolu se zprávami také shodné jméno i seznam parametrů. Množina všech zpráv, kterým objekt rozumí, tj. je schopen nalézt implementaci odpovídající metody, se nazývá *rozhraní objektu*³.

2.1.3 Základní koncepty OOP

Objektově orientované programování slučuje nové programovací koncepty a vylepšuje staré, aby tak dosáhlo přiblížení popisu reálného světa k lidskému způsobu uvažování. Tyto koncepty bývají často označovány za stavební kameny objektově orientovaného paradigmatu.

²Pojem atribut budeme upřednostňovat v obecnějším popise. V případě instanční proměnné zdůrazňujeme implementační přístup, tj. reference na odkazovaný/obsahovaný objekt.

³V literatuře se lze setkat s pojmem protokol objektu. My však pod pojmem *protokol* budeme chápat postup objektu při hledání reakce na přijatou zprávu.

- **Objekty** – spojují data a funkcionalitu společně do jednotek zvaných *objekty*, ze kterých se potom skládá výsledný objektově orientovaný program na rozdíl od strukturovaného složeného z procedur a funkcí. Objekty jsou tedy základní jednotkou modularity i struktury v objektově orientovaném programu, která umožňuje problém intuitivně rozdělit na přímo realitě korespondující podčásti a díky jejich vzájemné komunikaci i tento problém řešit.
- **Abstrakce** – neboli schopnost programu ignorovat/zjednodušit/zanedbat některé aspekty informací či vlastností objektů, se kterými program pracuje. Abstrakce je pohled na vybraný problém reálného světa a jeho počítačové řešení. Při vytváření takovéto abstrakce je vhodné mít možnost skrývat detaily do jakési černé skříňky (angl. black box), která je pro okolí definovaná pouze svým rozhraním, přes které komunikuje s okolím, a nikoli vnitřními detaily implementace, která může být podstatným zjednodušením reálného světa. Při konstrukci černé skříňky je dobré mít na paměti varianty a invarianty řešeného problému⁴.

Důležitým rozhodnutím je potom míra abstrakce, tedy jak hodně je vzdálená funkčnost černé skříňky od reality, respektive jak detailně potřebujeme realitu pomocí této abstrakce modelovat. Zde je hlavním limitujícím faktorem složitost a náročnost implementace perfektní černé skříňky, která by byla jednoduše použitelná, určená pouze svou funkcionalitou velmi blízkou reálnému chování abstrahovaného objektu či problému.

- **Zapouzdření** – zajišťuje již na úrovni definice sémantiky jazyka, že uživatel nemůže měnit interní stav objektů libovolným (tedy i neočekávaným) způsobem, ale musí k tomu využívat poskytované rozhraní (operace nad objektem). Každý objekt tedy nabízí protokol, který určuje, jak s ním mohou ostatní objekty interagovat (komunikovat). Ostatní objekty se tedy při komunikaci s tímto objektem spoléhají pouze na jeho externí (poskytované) rozhraní a skutečná implementace zůstává dokonale skryta. Právě tento koncept zásadně zjednodušuje vývoj nových vlastností a vylepšování stávající implementace našeho programu, protože nám stačí zachovat pouze zpětnou

⁴*Invariant* je taková část programu, že se hodnoty proměnných ani chování této části programu nemění při opakovaném provádění (průchodu) kódu této části programu. Zcela opačně je popsán *variant*, což je část programu, která hodnoty proměnných nebo svoje chování alespoň v některých průchodech mění.

kompatibilitu rozhraní objektů a nikoli skrytých implementací.⁵ Klíčovou roli hraje tato vlastnost také pro umožnění znovupoužitelnosti obecnějších objektů v různých projektech.

- **Polymorfismus** – neboli mnohotvárnost využívá mechanismus zasílání zpráv. Namísto běžného volání podprogramů (procedur a funkcí) ve strukturovaném programování se v OOP využívá mechanismu zasílání zpráv. Konkrétní použitá metoda reagující na zaslání zprávy závisí na konkrétním objektu, jemuž je tato zpráva zaslána. Například, pokud máme objekt `orel` a pošleme mu zprávu `rychle_se_přemísti`, tak implementace reagující metody bude pravděpodobně obsahovat příkazy pro roztažení křídel a vzletnutí. Pokud ale budeme mít objekt `gepard`, tak implementace metody volané při obdržení téže zprávy bude obsahovat například příkaz pro rozběhnutí se. Obě reakce na příjem stejné zprávy byly uzpůsobeny potřebám konkrétního objektu, který zprávu obdržel, a tudíž byly plně v jeho vlastní režii. Takto získáme *polymorfismus*, kdy ta samá proměnná programu může během jeho běhu obsahovat či odkazovat různé objekty, a tak může zaslání stejné zprávy objektu ve stejné proměnné při provádění stejné části kódu vyvolat během různých kontextů (časových bodů, obsahů proměnných či instančních proměnných, hodnot parametrů) rozdílné reakce (invokovat různé metody).
- **Dědičnost** – je způsob, jak implementovat sdílené chování. Nové objekty tak mohou sdílet a rozšiřovat chování těch již existujících bez nutnosti vše znovu reimplementovat⁶. Dědičnost se v praxi využívá ke dvěma účelům: (1) k indikaci, že nové chování specializuje jiné již existující chování⁷ a (2) pro sdílení kódu (implementace metod). Tato vlastnost je tedy velmi důležitá pro udržitelnost a rozšiřitelnost (robustnost) objektově orientovaných systémů.

První tři koncepty lze při dobré vůli a snaze programátora využít již v modulárním způsobu programování, ale až OOP nás nutí tyto zásady přísněji dodržovat. Některé hybridní objektově orientované jazyky jako C++ nám umožňují částečné obcházení některých těchto konceptů (např. veřejný modifikátor viditelnosti u instančních

⁵Některé OOP umožňují přísné zapouzdření instančních proměnných porušovat pomocí tzv. modifikátorů viditelnosti (viz sekce 5.3.3 na straně 66).

⁶Jazyky založené na objektech pracují místo dědičnosti s pojmem delegace.

⁷Ve staticky typovaných jazycích hraje vztah dědičnosti významnou roli při typové kontrole.

identita vs shoda

proměnných tříd; existence entit, které nejsou objekty nebo klasická volání procedur a funkcí obcházejících mechanismus zasílání zpráv).

Důležitou operací prováděnou nad objekty je *identita*. Identita porovnává, zda jsou objekty totožné, zda se jedná o tentýž objekt. *Shodnost* dvou objektů je operace, která provádí porovnání objektů podle jejich obsahu. Za shodné pak mohou být označeny i neidentické objekty. Matematicky řečeno, identita dvou objektů implikuje shodnost objektů, ale ne naopak.

Příklad 2.1.1 *Ukázka pojmu shoda a identita pomocí tří proměnných obsahujících dva objekty (objekt vytvoříme pomocí literálu | seznam atributů s hodnotami |).*

```
pavel := | vek = 10, vyska = 135 |
tomasuvSyn := martin := | vek = 10, vyska = 135 |
```

Objekt v proměnné `pavel` je shodný s objektem v proměnné `martin` i s objektem v proměnné `tomasuvSyn`, protože mají stejné hodnoty všech atributů.

Objekt v proměnné `pavel` není identický s objektem v proměnných `martin` respektive `tomasuvSyn`, protože se nejedná o tu samou část paměti, tj. změna atributu v objektu `pavel` nemá stejný efekt na objekt `martin`.

Ale objekt v proměnné `martin` je identický s objektem v proměnné `tomasuvSyn`.

2.1.4 Minimální model výpočtu

Minimální model výpočtu v čistě OOJ obsahuje dvě základní sémantické konstrukce.

přiřazení

První konstrukcí je pojmenování objektu neboli přiřazení objektu do proměnné. Místo samotného objektu se může pracovat s referencí⁸ nebo ukazatelem na daný objekt.

zaslání zprávy

Druhá konstrukce je zaslání zprávy objektu. Samotná zpráva kromě svého identifikátoru (jména) může obsahovat také parametry (další objekty upřesňující význam zprávy). Způsob reakce objektu na přijatou zprávu záleží již na něm samotném, což je důsledek konceptu zapouzdření a polymorfismu. U objektů jsou reakce na zprávy nejčastěji implementovány pomocí stejně pojmenovaných metod. Při obdržení zprávy příjemcem mohou nastat tři různé situace:

⁸Např. v jazyce C++ je reference speciální druh ukazatele, který ale nepovoluje obsahovat nedefinovanou hodnotu neboli neukazovat nikam.

- 1) Objekt ve své implementaci při reagování na zprávu nalezne a zavolá příslušnou metodu.
- 2) Objekt sice hledanou metodu přímo neobsahuje, ale obsahuje ji jiný objekt v rodičovském vztahu s tímto objektem. Přesný způsob, jakým se hledají žádané metody v objektu samotném a v objektech k němu vztažených, probíhá v závislosti na programovacím jazyce a je závislý na způsobu implementace konceptu dědičnosti a tzv. směrování zpráv.
- 3) Objekt implementaci odpovídající metody neobsahuje a ani nebyla nalezena v objektech předků, jak to popisuje předchozí případ. Pak nastává výjimka (chyba), že přijaté zprávě nebylo porozuměno, neboli že k ní neexistuje odpovídající metoda. U dynamicky typovaných (např. Smalltalk) a netypovaných jazyků je tento případ poměrně častým projevem nedokončené implementace chování daného objektu nebo špatného použití daného objektu. Staticky typované jazyky (např. C++, C# a Java) naopak dokáží tuto situaci většinou rozpoznat již při překladu a upozornit na ni chybovým hlášením.

Po vykonání metody, jež reagovala na zprávu, se řízení běhu objektového programu zpravidla vrací spolu s návratovou hodnotou (pokud nějaká je) do objektu odesílatele, kde se pokračuje ve výpočtu (v metodě, která zprávu odeslala). Takže z hlediska zpracování se metody chovají analogicky jako obyčejné funkce.

2.1.5 Výhody a nevýhody OOP

Následuje několikabodové shrnutí výhod a nevýhody OOP:

- + vyšší míra abstrakce
- + přirozenější práce se zapouzdřením a moduly (softwarový objekt je analogií/abstrakcí reálného objektu, a lze jej považovat za jakýsi samostatný modul)
- + jednodušší dekompozice problémů (rozdělení mezi abstrakci a zapouzdření)
- + udržitelnost a rozšiřitelnost (robustnost; změny mají vzhledem k objektům i třídám lokální charakter a vždy je jasné vymezeno, kde se všude mohou změny promítnout; eliminace postranních efektů)

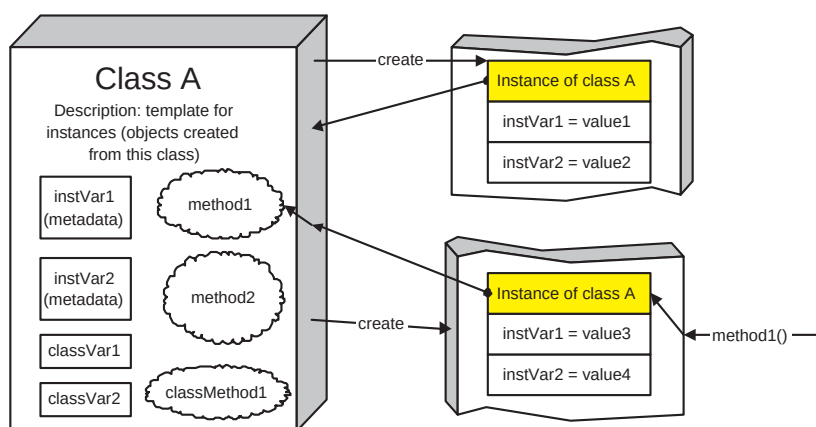
- + znovupoužitelnost (nejen koncept dědičnosti, ale i možnost využívat množiny cizích objektů při znalosti jejich rozhraní respektive protokolu pro komunikaci s nimi)
- v některých oblastech neexistuje analogie s reálnými objekty a pak je obtížné určit a definovat intuitivně ty softwarové
- složitější sémantika než u modulárních a strukturovaných jazyků vyžaduje více času na naučení
- nemožnost porušovat koncepci zapouzdření (v některých jazycích jako Java a C++ je to obcházeno pomocí modifikátorů viditelnosti)
- výsledný generovaný kód je pomalejší kvůli využívání dodatečných kontrol či odlišných modelů výpočtu než jsou vlastní von Neumannově architektuře počítače (virtuální objektová paměť a virtuální stroj, viz kapitola 5.4.2)
- režie na uložení objektů v paměti (např. odkaz na třídu objektu)

Pojmy k zapamatování

Objekt, zpráva, metoda, protokol, rozhraní, abstrakce, zapouzdření, polymorfismus, dědičnost

2.2 Datové a řídicí abstrakce

Nyní jsme si popsali objekt a jeho základní části. V případě, že budeme s objekty programovat, tak velice často budeme potřebovat, aby nějaká množina objektů rozuměla stejným zprávám, a nebo dokonce měla i stejnou reakci na přijaté zprávy (metody). První možnost, kterou máme k dispozici, je mít v každém objektu znovu přítomné ty samé metody, a případně i ten samý kód. To by ovšem značně znepříjemňovalo samotné programování, udržování programu a v neposlední řadě mělo obrovské paměťové nároky. Řešení, které využívá většina OoJ hlavního proudu (angl. *mainstream*) zavádí pojmy — třída a dědičnost. Obecně se těmito jazykům říká jazyky založené na třídách (angl. *class-based*), čili třídní jazyky. K tomuto existuje i alternativní beztřídní řešení, které popíšeme až na konci kapitoly a používá pojmy jako prototyp, rys a delegace. Tento přístup bývá využíván v jazycích založených na objektech (angl. *object-based*), neboli prototypových jazycích.



Obrázek 2.1: Třída je šablona pro tvorbu instancí a případně i zajišťuje reakce na zprávy zaslané těmito instancím.

2.2.1 Třídně orientované jazyky

Pokud máme v systému více objektů, které obsahují sice jiné hodnoty, ale stejné metody a shodnou vnitřní strukturu (množinu instančních proměnných), tak je výhodné pro takovéto objekty vytvořit speciální objekt, který nazveme *třída* a který slouží jako společný popis jejich vlastností (nikoli přímo hodnot atributů).

Definice 2.2.1 *Třída je šablona (otisk), podle níž mohou být vytvářeny objekty (instance této třídy). Třída se také stará o správu protokolu objektu, směřování zpráv a obsahuje implementace některých metod.* **Definice!**

Na třídu lze také nahlížet jako na metadata o objektu, která jsou zobecněním a zapouzdřením pojmu abstraktního datového typu zavedeného v předchozích kapitolách o modulárním programování.

Matematicky lze na pojem třída nahlížet jako na podmnožinu množiny všech objektů takovou, že její prvky mají něco společného. V našem případě mají společný protokol a vnitřní strukturu (případně další společná metadata).

Instanciace objektů

Proces samotného vytváření objektu pomocí předpisu daného konkrétní třídou nazýváme *instanciace*, nebo též vytvoření instance třídy. Instance třídy je objekt, který obsahuje naplněny instanční proměnné a odkaz na třídu, ze které vznikl (svým způsobem se jedná o typovou informaci, která je často nutná nejen při překladu, ale i za běhu programu).

Bezprostředně po vytvoření objektu (vyhrazení prostoru v paměti a naplnění odkazu na třídu původu do této paměťové struktury) dostáváme prázdný objekt, jehož datové položky (instanční proměnné) musejí být teprve naplněny, neboli inicializovány. K tomu je obvykle v dané třídě vyčleněna jedna nebo více metod a takovouto metodu označujeme jako *konstruktor*. Strategie volání konstruktorů se jazyk od jazyka liší a v některých případech je poměrně komplikovaná (např. pořadí volání konstruktorů v jazycích s vícenásobnou dědičností jako je C++). V mnoha jazycích se dodržuje zvyklost, že jméno konstrukturu odpovídá jménu třídy (např. Java, C++, C#), které patří, nebo má nějaké předem stanovené neměnné jméno pro všechny třídy (např. PHP5, Python).⁹

Pro operaci vytvoření objektu se používá klíčové slovo¹⁰ *new*, které obsahuje jako parametr jméno třídy, která pro něj bude sloužit jako konstrukční šablona, a případný seznam hodnot parametrů, které jsou předány konstrukturu (pokud jej daný OOI automaticky volá).

kopírování objektů

K tématu vytváření objektů patří i poznámka o dvou přístupech k vytváření kopií objektů (ať již na úrovni třídních nebo prototypových jazyků, kde hovoříme o tzv. klonování):

- hluboká kopie (angl. deep copy) — Kromě objektu jako takového (soubor atributů) jsou kopírovány i objekty, které ony instanční proměnné referencují. V případě provádění hluboké kopie je často potřeba si i určit, do jaké hloubky (úrovně) se kopírování provádí, jinak by mohlo dojít ke kopírování velké části objektové paměti.
- mělká kopie (angl. shallow copy) — Jednoduše řečeno se jedná o hlubokou kopii do hloubky nula. Je tedy vytvořen nový objekt, ale všechny instanční proměnné obsahují odkazy na totožné objekty jako kopírovaná předloha.

Manipulace s objekty v kódu a v paměti

Pokud nově vytvořený objekt přiřadíme do proměnné, lze rozlišovat dva případy:

- a) Jazyky jako Oberon nebo C++ rozlišují, zda je objekt alokovan na zásobníku či haldě a rozlišují tak mezi samotný

⁹Smalltalk umožňuje implementaci vlastní třídní metody vlastního názvu a tudíž lze zvolit i jméno konstrukturu; standardně *initialize*.

¹⁰nebo třídní zpráva (např. v Pythonu jde o třídní zprávu pojmenovanou jako samotná třída spolu s kulatými závorkami na případné parametry následně předávané konstrukturu)

datovým záznamem objektu (blok paměti obsahující hodnoty datových členů objektu) a pouhou referencí na objekt. Tento přístup je sice komplikovanější a vyžaduje hlubší programátorův zájem, ale na druhou stranu pak umožňuje provádět efektivnější optimalizace, a to již v době překladu.

- b) Naopak jazyky jako je Simula, Smalltalk nebo Modula-3 před programátorem tento rozdíl skrývají a pracují s objekty pouze přes reference. Tímto mechanismem lze ušetřit paměť nejen při provádění příkazu přiřazení, ale také při předávání parametrů (tzv. implicitní předávání odkazem).

Samotný objekt v paměti obsahuje pouze hodnoty instančních proměnných a metainformaci určující, z jaké vznikl třídy, která již zajišťuje poskytnutí dalších důležitých informací o objektu (metainformace), jako např. protokol objektu nebo umístění kódu metod.

Rušení objektů v paměti

V dnešní době se používají dva způsoby rušení instancí:

1. **Automaticky** — Automatické rušení je většinou možné pouze v případě běhu objektového prostředí ve virtuálním stroji a provádí jej nástroj zvaný *garbage collector*, který jednou za čas vyhledá objekty, na které již neexistuje žádný odkaz a ty zruší. Tímto postupem se likvidují i shluky objektů. Shluk objektů je množina objektů odkazovaných sice mezi sebou, ale žádným objektem mimo shluk. Těsně před samotným zrušením umožňuje většina jazyků volat specializovanou metodu pro úklid a uvolnění alokovaných zdrojů mimo objekt (např. připojení k databázi či otevřené soubory), tzv. *finalizace*. Záruka volání finalizační metody při destrukci objektu se liší jazyk od jazyka.
2. **Manuálně** — V případě, že nemáme k dispozici *garbage collector* (většinou u jazyků kompilovaných přímo do nativního kódu procesoru) nebo jej máme možnost nepoužít, tak je potřeba se o rušení objektů starat manuálně (tedy na úrovni zdrojového kódu). Pro likvidaci objektů bývá vyčleněna speciální metoda, tzv. *destruktor*, kterou když programátor zavolá, tak na jejím konci je objekt uvolněn z paměti. Kvůli dědičnosti musí tyto jazyky specifikovat pořadí volání destruktů (analogicky jako u konstruktorů). Další specialitou je tzv. *virtuální destruktor*, který má analogickou funkci jako virtuální metody (viz popis polymorfismu v sekci 2.2.1 na straně 29). V případě špatného uvolňování paměti, kterou má plně na zodpovědnost programátor, vznikají tzv. *úbytky paměti* (angl. *memory leaks*).

Dědičnost a hierarchie tříd

Z matematického pohledu dědičnost zavádí relaci „třída B dědí od třídy A “. Tato reflexivní, tranzitivní a antisymetrická relace tvoří částečné uspořádání na množině tříd a vyúsťuje v možnost uspořádání tříd, v případě jednoduché dědičnosti, do stromové hierarchie tříd. Pak třídě A říkáme *rodič* třídy B a naopak B je *potomek* A . Kořenem takového hierarchického stromu tříd je třída, která je předkem všech ostatních tříd. V listech jsou naopak třídy, které již nemají žádné potomky.

Účely dědičnosti

1. znovupoužití definované třídy pro specifitější verzi třídy (typu)
2. zajištění zpětné kompatibility z pohledu rozhraní instancí zděděných tříd

Klasifikace dědičnosti

Dvě základní kritéria klasifikace dědičnosti tříd jsou (A) „Kolik rodičů může mít potomek?“ a (B) „Co se dědí?“:

- (A₁) *Jednoduchá dědičnost* říká, že každý potomek má nejvýše jednoho přímého předka (rodiče). Např. jazyk Java, Smalltalk, C#.
- (A₂) *Vícenásobná dědičnost* je případ, kdy může třída dědit od více přímých předků (více jak jednoho). Do hry však přichází problémy s konflikty a duplikací jmen členů různých předků apod. Nejčastější řešení těchto problémů je zakázání výskytu konfliktů jmen členů nadtříd, případně vyžadování uvádění kvalifikace předka, ze kterého konfliktní metody invokovat¹¹. Další výzvou v jazycích s vícenásobnou dědičností je vytvoření optimálního a nejpoužitelnějšího algoritmu pro vyhledávání metod, které mají reagovat na zaslanou zprávu, kdy je nutno procházet orientovaný graf namísto jednoduchého stromu, jak je tomu v případě jednoduché dědičnosti.
- (B₁) Dědičnost neboli *dědičnost implementace* – kromě atributů jsou do dědičnosti zahrnuty celé metody včetně jejich implementace. Zde právě v případě vícenásobné dědičnosti vzniká problém s různými implementacemi dvou stejných metod nebo s různými definicemi dvou stejně pojmenovaných instančních proměnných.

¹¹Přesněji řečeno je třeba uvádět předka, ve kterém má začít hledání reakce na zprávu, která má v aktuální třídě více zděděných reakcí (konfliktních metod).

- (B₁) *Dědičnost rozhraní* – ze snahy o využívání vícenásobné dědičnosti, ale odstranění zmíněných problémů s konfliktními jmény, vzniká přístup dědění pouze na úrovni rozšiřování protokolu objektu. V praxi se jedná o předpis nebo seznam metod, které je nutno v potomkovi implementovat¹² a tudíž splňuje druhý účel dědičnosti – dohodu o rozhraní.¹³

Příklad 2.2.1 Ukázka tranzitivity dědičnosti

Mějme neprázdnou sekvenci tříd $C_1, \dots, C_n$, kde C_{i+1} přímo dědí od C_i pro všechna $i \in \{1, \dots, n-1\}$. Díky tranzitivitě dědičnosti tedy platí, že i třída C_n dědí od C_1 . Dále platí, že třída C_n je tzv. *podtyp* třídy C_1 ¹⁴.

V čistě objektově orientovaných třídních jazycích jsou všechny entity objektem a každý objekt má svou třídu¹⁵. Z tohoto jednoduchého konceptu plyne i to, že i třídy mají své třídy, které označujeme jako *metatřídy*. Většina třídních jazyků na tomto místě třídní hierarchie (kořenový uzel) již umožňuje vytvořit smyčku (viz příklad 2.2.2), takže třídou k metatřídě již většinou bývá samotná metatřída.

Příklad 2.2.2 *Příklad na třídě Time jazyka Smalltalk, které zasíláme zprávu class pro zjištění třídy, případně ještě zprávu name pro získání přesného názvu třídy (řetězec je v apostrofech). Podstatné jsou především poslední tři řádky kódu, které demonstrují cyklus v hierarchii dědičnosti.*

```
time := Time now.           // zpráva vyvolá: třídní metodu now
time class = Time.          // instanční metodu class
Time class name = 'Time class'. // třídní metodu class a name
Time class class = Metaclass.
Metaclass class name = 'Metaclass class'.
Metaclass class class = Metaclass.
```

Libovolný objekt má tedy instanční proměnné a třídu, která zajišťuje operace nad tímto objektem. V případě, že objekt není třídou¹⁶, mluvíme o instančních proměnných a metodách. Pokud je objekt třídou, mluvíme o třídních proměnných a třídních metodách. Instanční metody jsou implementací reakcí na zprávy, které obdržel objekt, a

¹²Dokud nemá potomek implementovány všechny požadované reakce na zprávy, nelze vytvářet jeho instance (viz obrázek 4.2 na straně 52).

¹³V praxi nemá smysl pracovat s jednoduchou dědičností rozhraní, ale čistě teoreticky je i tato kombinace možná.

¹⁴Formální zápis vztahu *podtyp* je zatím nejednotný, např.: $C_n > C_1$ vyjadřující, že C_n je specifitější než C_1 , tj. obsahuje více detailů; na druhou stranu zápis $C_n < C_1$ je odvozen od toho, že objekty podtypu C_n tvoří vždy podmnožinu (většinou vlastní) množiny instancí typu C_1 .

¹⁵Třída je také objekt.

¹⁶Každý objekt je instancí třídy.

jsou uloženy ve třídě tohoto objektu. Instanční proměnné jsou součástí přímo daného objektu. Třídní metody jsou reakce na zprávy zaslané třídám (např. `now` z příkladu 2.2.2). Třídní proměnné jsou uloženy ve třídách. Třídy jsou unikátně pojmenované instance metatříd.

V hybridně objektových jazycích zpravidla nebývají metatřídy z jazyka přístupné. Definice třídních proměnných a třídních metod je v tom případě zapisována přímo do definice třídy (nikoli metatřídy) a označena předepsaným klíčovým slovem (nejčastěji `static`). Proto statické proměnné a metody se potom třídním proměnných a třídním metodám říká *statické*.

Typy, podtypy, nadtypy

Hovoříme-li o třídě jako o typu (podobně jako abstraktní datový typ má i třída operace a vnitřní uložení dat), tak v případě předka této třídy hovoříme o *nadtypu* (angl. *superclass*) a u potomka o *podtypu* (angl. *subclass*). Tento vztah však obecně v druhém směru neplatí, jak bude demonstrováno v této sekci.

Příklad 2.2.3 *Jeden z motivačních příkladů pro zavedení dědičnosti ve staticky typovaných jazycích.* Mějme metodu (případně funkci) *m*, která jako parametr vyžaduje objekt třídy *A*. Pak jako parametr lze metodě *m* zadat:

1. objekt stejného typu (např. instanci třídy *A*);
2. objekt třídy odvozené od třídy *A* (např. pomocí dědičnosti).

Jednou z motivací pro zavedení dědičnosti ve staticky typovaných jazycích je umožnění zástupnosti typů, které jsou odvozeny pomocí dědičnosti od patřičných tříd (potažmo typů).

Je-li tedy instance třídy nebo z ní zděděné třídy vyžadována na místě parametru metody (příp. funkce), tak mluvíme o *vyžadované dědičnosti* (angl. *required inheritance*), kdy dědičný vztah zajišťuje existenci potřebného protokolu (nebo jeho nadmnožiny; např. existence patřičných atributů nebo virtuálních metod). Vyžadovaná dědičnost je zpravidla potřebná u staticky typovaných jazyků jako Java, C++ nebo Object Pascal z prostředí Borland Delphi.

skutečný podtyp Pokud potřebujeme více volnosti s dosazování za parametry metod, tak musíme využít jiný způsob zajištění existence metod (případně i atributů). Nejprůchoďejší je kontrola existence požadovaných metod (příp. atributů) v konkrétním dosazovaném typu, což budeme nazývat kontrola *skutečného podtypu* (angl. *real subtype*). Podobným způsobem je prováděna kontrola například v modulárním jazyce C při testování kompatibility struktur (heterogenní strukturovaný

Vyžadovaná dědičnost	Výsledek	Skutečné podtypy	Výsledek
call q(a);	OK	call q(a)	OK
call q(b);	chyba	call q(b)	OK
call q(c);	OK	call q(c)	OK
call p(b);	chyba	call p(b)	chyba
call p(c);	OK	call p(c)	OK

Tabulka 2.1: Rozdíl mezi vyžadovanou dědičností a kontrolou skutečného typu (sloupec Výsledek zachycuje výstup kontroly)

typ) během kompilace. Tato kontrola je obecnější než vyžadovaná dědičnost, která může být dědičností v některých případech omezena.

Příklad 2.2.4 Rozdíl mezi vyžadovanou dědičností a kontrolou skutečného typu (podtypu). Mějme deklarovány následující tři třídy *A*, *B*, *C*, takto:

```
class A is
  int d1;
  float d2;
  float f(int);
endOfClass

class B is
  int d1;
  float d2;
  float f(int);
  int g(float);
endOfClass

class C inherited
from A is
  int d3;
  float f(int);
  int g(float);
endOfClass
```

Nyní uvažujme použití objektů těchto tříd jako parametrů následujících dvou funkcí¹⁷:

```
int q(class A);
int p(class C);
```

Tabulka 2.1 potom demonstruje rozdílné chování obou druhů kontrol parametrů.

Nyní vidíme, že hlavní rozdíl spočívá v kontrole skutečné přítomnosti implementace potřebného protokolu (metod, atributů) (tzv. skutečný typ) oproti kontrole jména typu v případě vyžadované dědičnosti.

¹⁷Někdy je místo metod vhodnější příklady vlastností demonstrovat na funkcích, přestože jsou přítomné pouze v hybridně OO jazycích.

Z toho plyne, že nelze za všech okolností a ve všech OOO zcela bezmyšlenkovitě zaměňovat pojmy třída a typ (resp. podtřída/nadtřída a podtyp/nadtyp).

V třídních jazycích existují dva přístupy k typům:

- 1) Přístup **čistě objektový** (např. Smalltalk): Vše je objekt (pravdivostní, číselná hodnota i třída atd.) a existuje třída, nebo případně objekt, od kterého jsou všechny ostatní odvozeny.
- 2) Přístup **hybridně objektový** (podobný strukturovaným jazykům): Máme k dispozici sadu *základních, neboli primitivních typů* (pravdivostní hodnota, číslo, znak, řetězec), které lze případně skládat do homogenních nebo heterogenních struktur. Třída je potom případ heterogenní struktury, která může obsahovat kromě atributů i metody (funkce uvnitř této struktury). Při tomto přístupu rozlišujeme dva podpřípady:
 - a) Existuje *kořenová třída*, která je předkem každé existující nebo nové třídy (např. třída Object v Javě a C#, TObject v Object Pascalu, resp. Delphi)
 - b) Jazyky, které nemají v hierarchii dědičnosti žádnou kořenovou třídu (např. C++).

primitivní typy

Z praktického hlediska je vhodné optimalizovat přístup k primitivním datovým typům (číslo, řetězec) i v jazycích čistě objektových, ale vždy takovým způsobem, aby to nemělo žádný vliv na objektové chování těchto elementárních objektů.

staticky a dynamicky
typovaný jazyk

Staticky typovaný jazyk určuje množinu operací, které objekt podporuje, již v době překladu programu. V případě, že se zjistí, že po objektu je požadována nepodporovaná operace, neproběhne překlad úspěšně. *Dynamicky typovaný jazyk* pak tuto kontrolu provádí až v době běhu programu. V případě, že objekt požadovanou operaci nepodporuje, je proveden pokus o konverzi objektu na jiný typ, a případně vygenerována chyba. Další možností je, že je na tuto skutečnost upozorněn přímo objekt, který operaci nepodporuje. Detailnější klasifikace jazyků je v kapitole 5.2.

Po zavedení pojmu třída lze na dědění nahlížet i jako na vytváření podtypů a nadtypů. Vytváříme tak stromovou hierarchii tříd (typů), jež mohou vyčleňovat společné vlastnosti do nadtypů a specializovat své vlastnosti do podtypů.

Práce s metodami

redefinice metody

Redefinice metod (angl. *method overriding*) je možnost jazyka defi-

novat pro metodu podtřídy novou, specifitější implementaci, než je obsažena v její nadtřídě. Obě metody přitom mají stejnou signaturu (jméno metody, seznam formálních parametrů, návratový typ) jak v nadtřídě, tak v podtřídě. Implementace redefinované metody tedy nahradí implementaci, která se dědí z předka.

Redefinice metod je důležitý mechanismus OOJ, který obohacuje možnosti polymorfismu (viz sekce 2.2.1) a dědičnosti.

Poznamenejme, že některé OOJ na druhou stranu umožňují redefinici metody pomocí klíčového slova zakázat.

V každém OOJ je, ať už implicitním nebo explicitním způsobem, označován objekt příjemce zprávy. Toto pojmenování slouží k referencování příjemce v kontextu právě vykonávané metody (včetně konstruktoru nebo destruktoru). Nejčastěji se pro toto označení využívá pseudoproměnná označená klíčovým slovem `this` nebo `self`. Implicitní předávání příjemce zprávy metodě se většinou realizuje nultým skrytým parametrem této metody. Je dobré si také uvědomit, že příjemcem zprávy nemusí být vždy instance třídy, která metodu definuje, ale i instance jejího libovolného potomka.

V případě, že redefinujeme metodu a chceme využít kódu již definované metody (tj. pouze obalit již existující stejně pojmenovanou metodu novým kódem), tak potřebujeme mechanismus, jak se odkážeme na metody předka (případně předků). V jazycích s jednoduchou dědičností se daný požadavek řeší přidáním nového klíčového slova (např. `base` nebo `super`), které podobně jako v předchozím odstavci reprezentuje pseudoproměnnou obsahující náš objekt přetypovaný takovým způsobem, aby se tvářil jako instance rodičovské třídy. Při vícenásobné dědičnosti je situace komplikovanější a pro odkaz na metody v předcích, které již byly redefinovány, se používá specifikace konkrétní třídy (např. `A::puvodniMetoda()` v C++), tzv. kvalifikace jména.

Polymorfismus - časná a pozdní vazba

Polymorfismus umožňuje na místě jednoho objektu používat jiný bez nutnosti jakýchkoliv zásahů do programu. Bez jeho existence bychom museli programovat zvláštní kód pro každý možný typ objektu, se kterým hodláme komunikovat. To přirozeně není vzhledem k návrhu programu vůbec vhodné.

V praxi potřebujeme pracovat s objekty v obecné rovině, tzn. že jim zasíláme zprávy a ty na ně reagují podle vlastní potřeby, což odpovídá jisté míře autonomnosti objektů. Podle zásady zapouzdření je jen na vůli objektu samotného, jak se zprávou naloží. Při komunikaci s objektem nás zajímá pouze jeho veřejné rozhraní. Objekty, které toto

rozhraní mají podobné a pokrývají všechny námi požadované operace, můžeme považovat za zaměnitelné. Z toho je patrné, že polymorfismus jde ruku v ruce se zapouzdřením a obecně není nutně závislý na dědičnosti.

Příklad 2.2.5 Příklad polymorfismu pomocí vynucené dědičnosti

```
class Osoba is ... end;
subclass Student of Osoba is ... end;

var objOsoba : InstanceTypeOf(Osoba) := new Osoba;
var objStudent : InstanceTypeOf(Student) := new Student;
procedure f(x : InstanceTypeOf(Osoba)) is ... end;

objOsoba := objStudent;
f(objStudent);
```

časná vazba

pozdní vazba

U staticky typovaných jazyků se situace komplikuje. Při překladu programu totiž nemusí být vždy možné správně odhadnout, s jakým typem objektu budeme pracovat (*časná vazba* neboli statická vazba), a protože se požadovaná operace určuje již během kompilace, nemusíme zvolit vždy tu správnou. To se týká právě případů, kdy potřebujeme pracovat s objekty na abstraktní úrovni a konkrétní implementace operací nás nezajímá. V těchto případech je nutné přistoupit k identifikaci potřebné metody až v době běhu programu, kdy je ji nutné invokovat. Tento mechanismus se nazývá *pozdní vazba*, neboli dynamická vazba, a k jeho implementaci se většinou používají tzv. *tabulky virtuálních metod* (VMT, ilustrace viz sekce 5.4.1 na straně 76). Pak každá instance třídy, která obsahuje *virtuální metody* (metody umožňující pozdní vazbu), obsahuje odkaz na VMT. Tato tabulka obsahuje ukazatele na konkrétní metody, které jsou pro daný typ platné. Při překladu se pouze tyto tabulky sestrojí a určí, který řádek VMT se použije.

U dynamicky typovaných OoJ jsou všechny metody virtuální. Všechny objekty totiž obsahují přímo reference na třídu, jíž je objekt instancí, nikoliv pouze data (a odkaz na VMT). Metody příslušné ke zprávám se vyhledávají přímo v těchto třídách a jejich přítomnost není žádným způsobem kontrolována v době kompilace. V případě, že za běhu programu nastane situace, kdy nebyla nalezena implementace zprávy, dojde k výjimce¹⁸, kterou má samozřejmě možnost ošetřit daný objekt i sám programátor. Algoritmus používaný při hledání a výběru metody pro reagování na přijatou zprávu se označuje jako mechanismus *směrování zpráv*.

¹⁸Podrobnější informace ke konceptu výjimek jsou v sekci 5.3.7

2.2.2 Prototypově orientované jazyky

Nejdůležitějšími vlastnostmi objektově orientovaných jazyků jsou zapouzdření a polymorfismus. Díky těmto vlastnostem se objekty chovají jako uzavřené entity a v ideálním případě jen ony samy rozhodují o tom, jak naloží s přijatými zprávami od jiných objektů, tj. pracují autonomně.

Ve většině objektově orientovaných jazyků objekty delegují zprávy na svoje třídy, ovšem objektové paradigma dovoluje mnohem větší volnost a existuje celá řada programovacích jazyků, které pracují na základě mnohem méně tradičních konceptů.

Velice perspektivní skupinu objektově orientovaných jazyků tvoří jazyky založené na prototypech. Tyto jazyky unifikují svůj návrh tím, že znají pouze jediný typ objektů a nevyčleňují samostatně objekty reprezentující třídy. Rovněž objekty mají unifikovanou strukturu. Nyní se již nerozlišuje mezi instancemi, které obsahují nestatické proměnné, a třídami, které definují metody se sdíleným chováním, ale každý objekt může obsahovat jak členské proměnné, tak metody. Tyto složky objektu se označují jako *sloty*.

slot

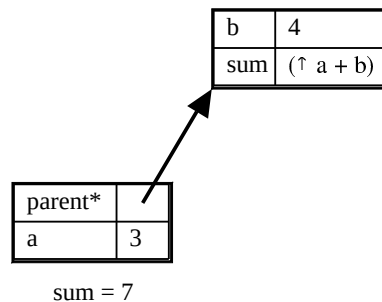
Sdílené chování

Pokud je objektu zaslána zpráva, prozkoumá množinu svých slotů a pokusí se najít ten, který poslané zprávě odpovídá. To znamená, že se použije hodnota instanční proměnné nebo dojde k invokaci odpovídající metody.

Nové objekty se vytvářejí kopírováním existujících objektů, tzv. *klonování*. Klonování objektu se implicitně provádí technikou mělkého kopírování. Teoreticky však může každý objekt přesně definovat všechny své operace, tzn. že i všechny příbuzné objekty by používaly své nezávislé kopie metod. Při změně kódu metody jednoho objektu by pak nedošlo ke změně korespondujících metod v objektech dalších. Tento přístup ovšem není příliš praktický, protože značně komplikuje hromadné změny chování u příbuzných objektů.

V praxi je vhodné sdílené chování příbuzných objektů definovat pouze na jednom místě. To je důvod, proč byly v třídních jazycích zavedeny třídy. Tuto operaci lze ovšem zajistit i bez přítomnosti tříd tak, že všechny podobné objekty budou delegovat své přijaté zprávy na stejné objekty.

V případě rušení objektů se objektově založené jazyky příliš neliší od třídních používajících *garbage collector*. Tj. pokud na objekt neexistuje žádná reference (vyjma reference z něho samotného), tak se provede jeho zrušení, včetně zrušení jeho slotů, což jsou také pouhé reference. Rušení je dále samozřejmě aplikováno i na objekty odkazo-



Obrázek 2.2: Ukázka dvou objektů (se dvěma sloty) v rodičovském vztahu

vané sloty zrušeného objektu v případě, že nejsou referencovány jiným objektem. Tímto lze likvidovat i celé shluky objektů.

Delegace

delegace

Delegace je proces velmi podobný obyčejnému zaslání zprávy. Na rozdíl od něj se ale při invokaci metody v delegovaném objektu nemění kontext. Tzn. že pokud je zpráva delegována na jiný objekt, je v tomto objektu nalezen odpovídající slot, a pokud je tímto slotem metoda, je spuštěn její kód s tím, že ukazatel na příjemce zprávy neodkazuje na delegovaný objekt, jak by tomu bylo u zaslání zprávy, ale na objekt, který delegaci provedl.

rodičovský slot

Pokud je objektu poslána zpráva a ten ve svých slotech nenalezne žádný vhodný, provede delegaci zprávy na objekty, které jsou odkazovány speciálním typem slotů označovaným jako *rodičovské sloty* (obrázek 2.2). Pokud ani rodičovský objekt neobsahuje vhodný slot, deleguje zprávu dále na rodičovské objekty, tedy objekty referencované rodičovskými sloty. I při této postupné delegaci referencuje odkaz na příjemce zprávy stále původního příjemce.

Třídy se sdíleným chováním tak lze nahradit objekty, které obsahují pouze metody (tzv. *rysy*). Instanciace je pak simulována klonováním objektů, jejichž rodičovské sloty odkazují tyto rysy. Nové objekty pak odkazují své rysy ve svých rodičovských slotech.

Dědičnost

rys

Pro objekty, které obsahují sdílené chování, a tím tak nahrazují nebo doplňují třídy, se používá označení *rys* (angl. *trait*). Provázáním jednotlivých rysů rodičovskými sloty lze dosáhnout obdoby dědičnosti. Více specializovaný rys pouze pomocí rodičovského slotu odkáže obecněji definovaný rys, takže na něj deleguje neznámé zprávy.

Některé jazyky založené na prototypch dovolují umístit do objektů více rodičovských slotů. Tímto způsobem dokáží elegantně vytvořit obdobu vícenásobné dědičnosti.

Protože hodnoty rodičovských slotů lze dynamicky měnit, lze dosáhnout velmi zajímavého chování, které se v ostatních objektových jazycích velmi špatně napodobuje. Jedná se o *dynamickou dědičnost*. Objekt má možnost v průběhu svého života bez omezení měnit své chování. Statická typová kontrola v takovýchto podmínkách samozřejmě naprosto ztrácí význam a tyto programovací jazyky jsou typované dynamicky.

V praxi se příliš často nesetkáme se situací, kdy nějaký objekt zcela změní svoje chování, ale případy, kdy je vhodné objekt dynamicky rozšířit o nové vlastnosti, jsou poměrně časté. Prototypově orientované jazyky se díky své dynamičnosti obejdou i bez použití sofistikovaných návrhových vzorů, resp. jejich implementace je triviální.

Prototypy

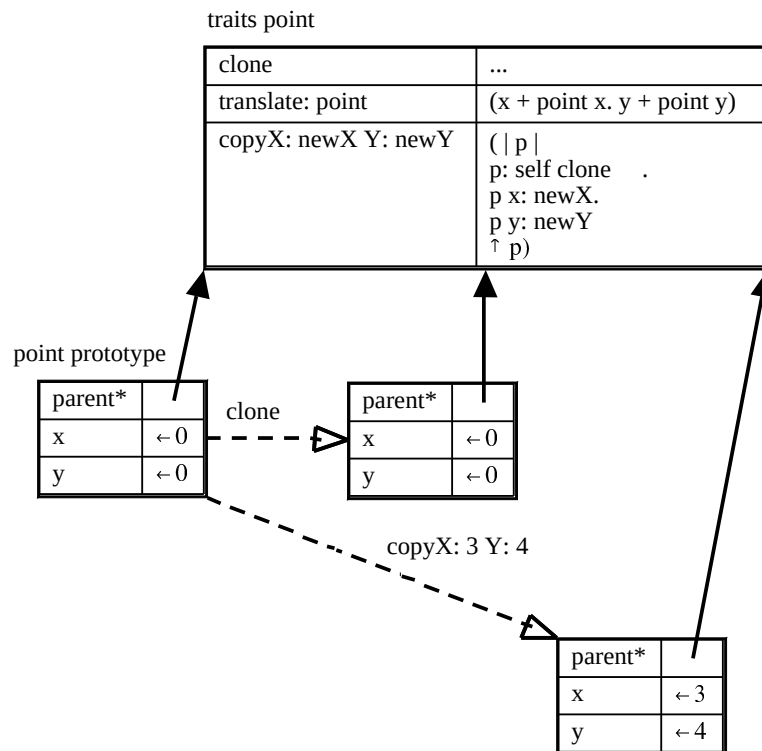
K tomu, abychom dosáhli plné náhrady funkce tříd ovšem samotné rysy nestačí. Problém je s instančními proměnnými, které jsou součástí samotných objektů a každý objekt musí mít jejich samostatnou kopii. Tento problém lze řešit více způsoby. Některé jazyky instanční proměnné přidávají do objektu a naplňují je standardními hodnotami v konstruktoru rysu. Takto činí například JavaScript.

Druhou možností je rys doplnit ještě jedním objektem, který obsahuje sloty instančních proměnných s implicitními hodnotami a ve svém rodičovském slotu odkazuje rys. Nová instance se pak vytvoří nakopírováním této šablony (viz obrázek 2.3). Šablony instancí se označují jako *prototypy*. Od nich tato třída jazyků také dostala své prototyp jméno.

První varianta je vhodnější pro jazyky orientované na zdrojové texty, druhá varianta je efektivnější v jazycích s vizuálně orientovaným vývojovým prostředím.

V syntaxi prototypově orientovaných jazyků nenalezneme pojmy jako rozhraní, mixins apod., ovšem dokáží je více než plně nahradit svými stávajícími prostředky bez toho, aby pro ně musely vytvářet zvláštní syntaktické konstrukce.

Obecně platí, že jazyky založené na prototypch nemají tak bohaté výrazové prostředky, jako jiné objektové jazyky, ovšem svými schopnostmi je díky své dynamičnosti často převyšují. Jejich prvním představitelem byl jazyk SELF, který byl navržen v roce 1986 jako dialekt Smalltalku založený právě na prototypch. Dnes v praxi nejčastěji používaným zástupcem této skupiny je již zmiňovaný JavaScript, který



Obrázek 2.3: Ukázka rysu a prototypu, včetně použití (jazyk SELF)

se však v dalších ohledech od jazyka SELF značně odlišuje. Jazyky založené na prototypu v sobě skrývají velký potenciál. Jsou především velmi vhodné a názorné při vizuálním modelování, prototypování a programování.

Nevýhodami těchto jazyků je vyšší režie (hlavně paměťová náročnost) a podstatně obtížnější vyhledávání objektů se stejným protokolem. Podrobnější představení vlastností řady prototypových jazyků naleznete v [22].

Pojmy k zapamatování

třída, instance, dědičnost, brzká (časná) a pozdní vazba, volání/invokace metody, slot, rys, prototyp, delegace, klonování

Úlohy k procvičení:

Vyberte si libovolný OoJ a prostudujte jeho syntaktické i sémantické možnosti a vlastnosti. Popište, které vlastnosti zde popsány mají a které mu chybí, jak by byl klasifikován a jaké má výhody a nevýhody.

2.3 Závěr

Objektově orientované programování je v současnosti velmi dobrá volba pro většinu metodologií při tvorbě počítačových aplikací. Hlavním přínosem je zjednodušení problému dekompozice větších problémů na snadněji řešitelné podproblémy a dobrá reflexe reality přes dostatečně přesné abstrakce. Objektově orientované jazyky jsou vhodné především pro větší a velké projekty, kdy se již daň v podobě režie na objektový způsob práce s daty, pamětí a operacemi vyplatí. Velkým plus je také dobrá robustnost (udržovatelnost) a flexibilita. V budoucnu mohou být OOJ nahrazeny jazyky s lepší implementací přímé podpory komponentních technologií či architektur orientovaných na služby.

Závěr

Objektově orientované paradigma je postaveno na komplexní datové struktuře—objekt, která sdružuje data a operace nad nimi do jednoho celku. Další koncepty jsou abstrakce, zapouzdření, polymorfismus a dědičnost, které přispívají ke správnému používání OO jazyka z pohledu softwarového inženýrství.

Model výpočtu se skládá pouze z příkazu (1) přiřazení a (2) zaslání zprávy, na kterou objekt reaguje spuštěním metody (implementace reakce na obdrženou zprávu) nebo vyvoláním výjimky o neporozumění zprávě. Zprávy, kterým objekt rozumí tvoří jeho protokol (rozhraní).

Podle způsobu vytváření nových objektů a podle způsobu implementace konceptu dědičnosti rozlišujeme dva základní typy OOJ—(1) třídně založené jazyky a (2) objektově založené jazyky, které tvoří minoritní skupinu a zabývá se jimi pouze kapitola 2.2.2 (pojmy delegace, slot, rys).

Zbýlý text se soustředí na vlastnosti třídně založených jazyků, jako způsob vytváření/rušení objektů, dědičnost (jednoduchá, vícenásobná, dědičnost rozhraní) a polymorfismus (časná vazba při překladu versus pozdní vazba za běhu programu).

2.4 Studijní literatura

Další zdroje

Jen málokdy se literatura zabývá pouze objektovým programováním samotným, ale většinou během výkladu základních principů zároveň probírá syntaxi a sémantiku vybraného jazyka. Následující seznam studijní literatury je příkladem několika takovýchto zajímavých pramenů:

- Merunka, V.: Objektové metody a přístupy - Smalltalk-80, učební text.

- Prata, S.: Mistrovství v C++, Computer Press, 2004, ISBN: 80-251-0098-7.

- Eckel, B.: Thinking in Java, 3rd Edition, Prentice-Hall, 2002.

Dostupné na URL <http://www.mindview.net/Books/TIJ/> (únor 2006)

- Křivánek, P.: Squeak Smalltalk, 2003-2005.

Dostupné na URL <http://www.comtalk.net> (únor 2006)

- Švec, M.: Programovací jazyk a aplikační prostředí SELF, FIT VUT Brno, 2004.

Dostupné na URL <http://www.fit.vutbr.cz/study/courses/TJD/public/0304TJD-Svec.pdf> (únor 2006).

Jak již bylo předesláno dříve, tak vysvětlení mnoha pojmů lze nalézt také zde:

- Wikipedia: Object-oriented programming, the free encyclopedia, 2005.

Dostupné na URL http://en.wikipedia.org/wiki/Object-oriented_programming (listopad 2005).

Kapitola 3

Formalismy a jejich užití

Tato kapitola seznamuje s možnostmi základních formálních prostředků pro popis syntaxe a sémantiky objektově orientovaných jazyků.

Čas potřebný ke studiu: 3 hodiny 30 minut.

Cíle kapitoly

Hlavní cíl této kapitoly je seznámit čtenáře se základními formálními prostředky, které se používají při popisu syntaxe a sémantiky objektově orientovaných jazyků. Pro ukázkou formálního popisu OOJ si definujeme jednoduchý netypovaný ς -kalkul (čti sigma kalkul). Pochopení ς -kalkulu je dostačující na pasivní úrovni, tedy se schopností interpretovat jednoduché výrazy tohoto modelu a chápat jejich význam. Nakonec naznačíme vztah grafického modelovacího jazyka UML s objektově orientovanou analýzou (OOA) a návrhem (OOD).

Průvodce studiem

Studium této kapitoly vyžaduje znalost pojmů jako je Backus-Naurova forma pro popis syntaxe programovacího jazyka a základní znalosti týkající se popisu formálních modelů (formální jazyky, logika). Text obsahuje matematicky náročnější pasáž doplněnou jednoduchými ilustračními příklady.

Obsah

3.1	Formální základ popisu objektově oriento-	
	vaného jazyka	39
3.2	ς-kalkul	39
3.2.1	Syntaxe a sémantika ς -kalkulu	40
3.2.2	Příklady	42
3.3	UML - formální vizuální jazyk	44
3.3.1	Výhody a nevýhody formálního návrhu	45
3.4	Závěr	45

Výklad

3.1 Formální základ popisu objektově orientovaného jazyka

Podobně jako u modulárních jazyků i u OOJ není formální základ nezbytný pro jeho návrh a v minulosti býval opomíjen. Z imperativních OOJ můžeme jako příklad jazyka s formálním základem uvést jazyk OCAML (odvozený od ML), který se úzce váže na jazyk SML s již existujícím formálním popisem. Tyto jazyky většinou obsahují i vlastnosti známé z funkcionálního programování.

Mnohem příznivější situace z tohoto hlediska je u deklarativních jazyků, kde je formální základ poměrně dobře rozpracován v podobě různých objektových kalkulů (netypovaných i typovaných), například tzv. ζ -kalkul a jeho varianty.

Formální základ lze opět rozdělit na dvě popisované oblasti — syntaxe a sémantika. Popis syntaxe OOJ využívá prostředků známých již z předchozích programovacích paradigmat, tedy především kombinace (E)BNF, bezkontextových gramatik a slovního výkladu doplněného o příklady. Tato nenáročnost je dána podobnou syntaktickou strukturou, kdy OOJ byly doplněny jen o několik nových klíčových slov a v některých jazycích nebylo nutné ani toto. Naopak sémantika jazyka doznala zásadních a rozsáhlých změn či spíše zavedení úplně nových významů a množství kombinací. Bohužel ve většině OOJ (především těch imperativních) je velmi obtížné složitou sémantiku formálně popsat, a proto se většinou využívá pouze vícevrstvý slovní popis s množstvím demonstračních příkladů. Hlavním úskalím je, že mohou zůstat utajeny některé okrajové sémantické vlastnosti jazyka, když se na ně v textovém popisu pozapomene nebo jejich vyjádření je pro čtenáře nejednoznačné.

3.2 ζ -kalkul

V následujících několika odstavcích se budeme zabývat jednoduchou variantou čistého netypovaného objektového kalkulu (symbol ζ čti jako *sigma*), abychom názorně demonstrovali příklad formálního základu objektově orientovaného jazyka.

3.2.1 Syntaxe a sémantika ς -kalkulu

Na objekty budeme nahlížet jako na primitivy s definovanou sémantikou.

Objekt se skládá z atributů (metod a paměťových buněk pro hodnoty). Nad každým objektem je potom nutno implementovat minimálně dva typy sémantických akcí — výběr atributu (exekuce či invokace metody/návrat hodnoty uložené v objektu) a změnu atributu (změnu těla metody/změna hodnoty v paměťové buňce objektu).

Syntaxe ς -kalkulu

ς -kalkul obsahuje minimální množinu syntaktických konstrukcí a výpočetních pravidel při zachování výpočetní úplnosti.

V celé této podkapitole budeme používat symbol $\triangleq$ pro ekvivalenci podle definice a symbol $\equiv$ pro syntakticky identické termy.

Každý program či výraz v ς -kalkulu je popsán jako ς -term, který vzniká z ς -podtermů následujícího tvaru:

Nechť $a, b_1, \dots, b_n$ jsou ς -termy, pak i následující zápisy jsou ς -termy:

1. Proměnná

$$x$$

2. Vytvoření objektu (obsahuje jen metody bez parametrů s návěštími l_i ; jediný a povinný formální parametr x_i slouží pro pojmenování objektu příjemce zprávy¹ s rozsahem platnosti v bezprostředně následujícím těle metody)

$$[l_i = \varsigma(x_i)b_i^{i \in 1..n}],$$

kde² každé l_i a l_j jsou navzájem různé pro $i \neq j$, $n \geq 0$ a

kde $\varsigma(x_i)b_i$ je metoda s pojmenováním objektu vlastníka metody návěští x_i a s tělem této metody b_i , kde je x_i vázanou proměnnou.

3. Invokace metody³ (spuštění metody) / výběr atributu

$$a.l$$

4. Modifikace metody / modifikace atributu (s funkcionální sémantikou, tj. výsledkem je nový objekt se změněnou definicí patřičné metody)

$$a.l \Leftarrow \varsigma(x)e'$$

¹V praxi se objekt příjemce zprávy označuje v kódu vlastních metod pseudo-proměnnou nebo klíčovým slovem *this* nebo *self*.

²Pro zkrácení zápisu obecného objektu budeme používat $l_i^{i \in 1..n}$ pro vyjádření posloupnosti $l_1, l_2, \dots, l_n$.

³ ς -kalkul nepracuje s mechanismem zasílání zpráv.

Příklad 3.2.1 Ukázka definice objektu pomocí ς -kalkulu:

$$o = [l_1 = \varsigma(x_1) [], l_2 = \varsigma(x_2)x_2.l_1]$$

Objekt označený o obsahuje dvě metody l_1 a l_2 , kde první vrací prázdný objekt $[]$ a druhá invokes metodu l_1 .

V příkladu 3.2.1 vidíme, že jediné operace, které lze s objekty provádět, jsou invokace metody jejího hostovského objektu (tečková notace pro volání metody objektu) a nebo její modifikace (symbol $\Leftarrow$). A až na proměnnou a literál pro zápis objektu (v hranatých závorkách) není v ς -kalkulu už žádná jiná syntaktická konstrukce. Syntaxi ještě doplníme kulatými závorkami pro explicitní vyjádření priorit a rozsahu platnosti jednotlivých proměnných. Také poznamenejme, že nezáleží na pořadí definice metod při zápisu literálu objektu. Písmeno ς je použito pro navázání hostovského objektu metody na proměnnou uzavřenou v následujících závorkách. Syntaktická konstrukce $\varsigma(x)b$ značí vázání proměnné x na rozsah podtermu b . V nejednoznačných situacích se vázání proměnné vztahuje na největší možný podterm bezprostředně vpravo od $\varsigma(x)$. Tedy například, $\varsigma(x)x.l$ znamená $\varsigma(x)(x.l)$ a ne $(\varsigma(x)x).l$.

Pro formální popis primitivní sémantiky ς -kalkulu potřebujeme volná a vázaná definice pojmu *volná* a *vázaná proměnná* v libovolném ς -termu. Volnost proměnné určuje rozsah platnosti této proměnné. Proměnná může být v daném výrazu buď volná nebo vázaná, ale ne obojí zároveň.

Mějme konečnou množinu proměnných $\mathcal{P}$, množinu všech platných výrazů ς -kalkulu $\mathcal{E}$ a funkci $FV: \mathcal{E} \rightarrow 2^{\mathcal{P}}$ počítající množinu volných proměnných ze zadaného výrazu definovanou takto:

1. $FV(\varsigma(y)b) \triangleq FV(b) - \{y\}$
2. $FV(x) \triangleq \{x\}$
3. $FV([l_i = \varsigma(x_i)b_i]^{i \in 1 \dots n}) \triangleq \bigcup_{i \in 1 \dots n} FV(\varsigma(x_i)b_i)$
4. $FV(a.l) \triangleq FV(a)$
5. $FV(a.l \Leftarrow \varsigma(x)b) \triangleq FV(a) \cup FV(\varsigma(x)b)$

Dalším dílčím mechanismem využitým při popisu sémantiky ς -substituce kalkulu je substituce (nahrazení) termu c za všechny volné výskyty proměnné x v libovolném ς -termu b , značíme $(b\{x \leftarrow c\})$, a definujeme ji takto:

1. $(\varsigma(y)b)\{x \leftarrow c\} \triangleq \varsigma(y')(b\{y \leftarrow y'\}\{x \leftarrow c\})$ pro $y' \notin FV(\varsigma(y)b) \cup FV(c) \cup \{x\}$

2. $x\{\!\{x \leftarrow c\}\!\} \triangleq c$
3. $y\{\!\{x \leftarrow c\}\!\} \triangleq y$ pro $y \neq x$
4. $[l_i = \varsigma(x_i)b_i^{i \in 1..n}]\{\!\{x \leftarrow c\}\!\} \triangleq [l_i = (\varsigma(x_i)b_i)\{\!\{x \leftarrow c\}\!\}^{i \in 1..n}]$
5. $(a.l)\{\!\{x \leftarrow c\}\!\} \triangleq (a\{\!\{x \leftarrow c\}\!\}).l$
6. $(a.l \Leftarrow \varsigma(y)b)\{\!\{x \leftarrow c\}\!\} \triangleq (a\{\!\{x \leftarrow c\}\!\}).l \Leftarrow ((\varsigma(y)b)\{\!\{x \leftarrow c\}\!\})$

Přejmenování y za y' a podmínka pro y' v bodě 1 nám reflektují přirozené pravidlo, že žádná volná proměnná x se substitucí nesmí stát vázanou proměnnou.

primitivní sémantika
 ς -kalkulu

Výpočet libovolného ς -termu se zapisuje formou posloupnosti redukčních kroků. Pokud se tedy b redukuje v jednom kroku na c , tak píšeme $b \mapsto c$.

Nechť $o \equiv [l_i = \varsigma(x_i)b_i^{i \in 1..n}]$ je objekt s navzájem různými návěštními všech jeho metod.

$o.l_j \mapsto b_j\{\!\{x_j \leftarrow o\}\!\}$, ($j \in 1..n$) je tzv. *redukce invokace* a

$o.l_j \Leftarrow \varsigma(y)b \mapsto [l_j = \varsigma(y)b, l_i = \varsigma(x_i)b_i^{i \in (1..n) - \{j\}}]$, ($j \in 1..n$) je tzv. *redukce modifikace*.

3.2.2 Příklady

Prvním příkladem je redukce objektu s jedinou metodou s návěštním l , jejímž tělem je prázdný objekt $[]$, a jedná se tedy z praktického hlediska o datový člen objektu:

Mějme objekt $o_1 \triangleq [l = \varsigma(x)[]]$. Při invokaci metody l v tomto ς -kalkulu se provede následující redukce $o_1.l \mapsto []\{\!\{x \leftarrow o_1\}\!\} \equiv []$, a nebo při modifikaci metody se například $o_1.l \Leftarrow \varsigma(x)o_1 \mapsto [l = \varsigma(x)o_1]$ a vznikne nový objekt s upravenou metodou l .

S využitím substituce „sama-za-sebe“ lze zkonstruovat i nekonečný výpočet, a to bez explicitního použití rekurze:

Mějme objekt $o_2 \triangleq [l = \varsigma(x)x.l]$, který při invokaci metody l bude donekonečna provádět redukci $o_2.l \mapsto x.l\{\!\{x \leftarrow o_2\}\!\} \equiv o_2.l \mapsto \dots$

V ς -kalkulu může metoda také například vracet objekt, ve kterém byla invokována:

Mějme objekt $o_3 \triangleq [l = \varsigma(x)x]$, pak $o_3.l \mapsto x\{\!\{x \leftarrow o_3\}\!\} \equiv o_3$.

Ze známých praktických jazyků má k tomuto formálnímu modelu velice blízko např. jazyk SELF, který taktéž nerozlišuje mezi datovými položkami a metodami objektu, ale zavádí jednotný pojem *slot*.

Další dva příklady demonstrují možnosti ς -objektů modifikovat své vlastní členské metody.

Mějme tedy např. $o_4 \triangleq [l = \varsigma(y)(y.l \Leftarrow \varsigma(x)x)]$, pak $o_4.l \mapsto (y.l \Leftarrow \varsigma(x)x)\{\{y \Leftarrow o_4\}\} \equiv (o_4.l \Leftarrow \varsigma(x)x) \mapsto [l = \varsigma(x)x] \equiv o_3$.

Poslední příklad ilustruje techniku zálohování obsahu objektu (metoda *backup*) před jeho změnou a následující možnost obnovy stavu objektu z této zálohy (metoda *retrieve*).

$$o \triangleq [\begin{array}{l} \text{retrieve} = \varsigma(s_1)s_1, \\ \text{backup} = \varsigma(s_2)s_2.\text{retrieve} \Leftarrow \varsigma(s_1)s_2, \\ \dots \end{array}]$$

Metoda pro obnovení (*retrieve*) je inicializována tak, aby vracela objekt samotný, protože zatím nebyla provedena žádná záloha. Dalším možným řešením by bylo vracet v takovém případě například prázdný objekt. Všimněme si, že *backup* ukládá objekt obsažený ve formálním parametru s_2 , který v daný okamžik obsahuje objekt aktuální v době invokace metody *backup*. Kdežto v době volání *retrieve* je aktuální objekt pojmenovaný s_1 . *Retrieve* tedy vrací objekt přesně, jak byl naposledy zazálohován.

$$\begin{aligned} o' &\triangleq o.\text{backup} = [\text{retrieve} = \varsigma(s_1)o, \dots] \\ o'.\text{retrieve} &= o \end{aligned}$$

Protože tato záloha opět obsahuje předchozí zálohu, lze se kaskádovým voláním propracovat k libovolné záloze.

Složitější příklady, jako definice objektově orientovaných přirozených čísel pomocí ς -kalkulu nebo příklad kalkulačky, vyžadují pro pochopení další rozšíření vlastností (jako například propracovanější operační sémantiku a transformace z λ -kalkulu; viz kapitola o funkcionálním programování). Dále lze ς -kalkul rozšířit o další vlastnosti známé z většiny OOJ, jako je dědičnost, třídy, typy, atd.

Úlohy k procvičení:

1. Z hlediska syntaxe i vyjadřovacích schopností porovnejte λ -kalkul známý z funkcionálního programování (viz text modulu o funkcionálním programování) a ζ -kalkul, který byl popsán v této kapitole.
2. Dále doplňte popis zde uvedených příkladů a výrazů ζ -kalkulu, například o určení volných a vázaných proměnných, upřesnění uskutečněných substitucí.
3. Nakonec se pokuste vymyslet vlastní jednoduchý správný výraz ζ -kalkulu, který se v tomto textu nevyskytuje a pokuste se jej vyhodnotit (redukovat).

Pojmy k zapamatování

ζ -kalkul, vytvoření objektu, invokace metody, modifikace metody, substituce, primitivní sémantika

Klíč k řešení úloh

K prvnímu bodu předchozích úloh řekněme, že libovolný výraz ζ -kalkulu lze převést na ekvivalentní výraz λ -kalkulu, což se často využívá například při formální verifikaci, kdy tak máme k dispozici větší výběr z podpůrných nástrojů.

3.3 UML - formální vizuální jazyk

Jedním z možných formalismů využívaných při objektově orientované analýze (OOA) a návrhu (OOD) je grafický jazyk UML, jehož syntaxe a částečně i nejčastěji používaná sémantika bude stručně zopakována v následující kapitole 4 s důrazem na popis entit a vlastností týkajících se OOP. Tento formalismus tedy není přímo užíván pro popis syntaxe nebo sémantiky objektově orientovaného programovacího jazyka, ale spíše objektových modelů a návrhů systémů. Přímo ve specifikaci UML je zřetelně naznačeno, že UML v žádném případě nesměřuje k nahrazení klasických programovacích jazyků zaměřených na zdrojové texty. Hlavním důvodem je silně abstraktní sémantika většiny konstrukcí v UML a obtížné vyjadřování velmi specifických vazeb, které se efektivněji dají vyjádřit prostřednictvím algoritmického textového zápisu.

Jeden z hlavních důvodů, proč je výhodné zavádět různé formalismy obecně (i v OOP) je vidina automatické nebo částečně auto-

matické formální verifikace. Problémy verifikace lze pak převést na problém zápisu celé aplikace prostřednictvím určitého matematického formalismu (např. ζ -kalkul). Toto je však mnohdy velmi náročný úkol.

Druhým méně náročným požadavkem je využívat formalismus na úrovni návrhu (angl. *formal design*) a ověřovat pouze správnost na této úrovni. Problémem je pak pouze zajištění správnosti transformace onoho návrhu do výsledné implementace.

Na druhé variantě staví většina procesů OOA a OON využívajících UML a nástroje pro formální verifikaci (příp. validaci) navržených UML modelů.

Většina modelů UML je reprezentována grafickými diagramy, z nichž stěžejní jsou diagramy tříd, objektů (instancí), aktivity, stavové, případů použití a mnoho dalších.

3.3.1 Výhody a nevýhody formálního návrhu

Jazyk pro popis návrhu je nezávislý na implementačním jazyce, obsahuje konstrukce pro popis struktury systému a komunikace mezi elementy systému. Takovéto návrhy často slouží jako vstup pro automatické generátory zdrojového kódu, a jak již bylo nastíněno, také jako vstup pro verifikační nástroje.

V tomto okamžiku přichází do hry problémy s udržováním/reflexí změn ve zdrojových kódech také v korespondujících formálních modelech, což je často neautomatizovatelný úkol.

Na druhou stranu jazyk jako například UML 2.0 není výpočetně úplný. Neklade si totiž za cíl nahradit současné implementační jazyky, ale zaměřuje se na popis specifikace, návrhu a samotného procesu vývoje. Kvůli vysoké míře abstraktnosti UML se také velmi rozcházejí různá rozšíření třetích stran (např. kompletní generátory zdrojových kódů nebo doplňky pro popis systémů pracujících v reálném čase).

3.4 Závěr

Kapitola nastínila čtenáři možnosti formálního popisu objektově orientovaných jazyků včetně jejich sémantiky. Pro pokročilejší studenty, také nastínila vztah mezi ζ -kalkulem a čistě objektově orientovanými jazyky, které jsou založené na objektech (prototypově orientované, viz sekce 2.2.2). Probíraný ζ -kalkul byl doplněn řadou jednoduchých příkladů pro podporu pochopení matematicky náročné látky.

Závěr

V úvodu kapitoly je zmíněn význam formálního základu jazyka pro možnosti automatické verifikace. Jako reprezentant je zvolena jednoduchá varianta ζ -kalkulu zapisující objekty jako soubory metod (uzavřené do hranatých závorek). Metody jsou definovány jako ζ -termy, které mohou být redefinovány jiným ζ -termem nebo invokovány. Všechny metody mají právě jeden formální parametr. Na tento formální parametr je při invokaci či modifikaci metody navázán objekt vlastníci danou metodu.

V textu je dále uvedena definice substituce a funkce pro určení volných (zbylé jsou vázané) proměnných ve výrazu ζ -kalkulu. Poslední teoretičtější část obsahuje popis primitivní sémantiky, jež je demonstrována na komentovaných příkladech.

Jako alternativní formální popis při objektově orientované analýze a návrhu lze využít populární vizuální jazyk UML.

Další zdroje

Knih obsahujících srozumitelně popsanou teorii o formálním popisu objektově orientovaných jazyků příliš mnoho nenajdete a většinou se jedná o anglicky psané publikace téměř na vědecké úrovni:

- Abadi, M., Cardelli, L.: A Theory of Objects, Springer, New York, 1996, ISBN 0-387-94775-2.

Literatura týkající se UML je součástí zdrojů k následující kapitole.

Kapitola 4

UML

Tato kapitola přehledově opakuje základní syntaxi a nejčastější sémantiku modelovacího jazyka UML, který se v současnosti intenzivně využívá při analýze, návrhu a dokumentaci objektově orientovaných systémů (aplikací).

Čas potřebný ke studiu: 3 hodiny 30 minut.

Cíle kapitoly

Hlavní cílem této kapitoly je poskytnout studujícímu ucelené a stručné informace o jazyku UML a jeho nejčastěji využívaných možnostech. Celá kapitola je psána formou rešerše shrnující jinak velmi objemný popis syntaxe, sémantiky a technik používání UML. Největší důraz je kladen na reprezentaci pojmů a vztahů, které jsme zavedli a vysvětlili v kapitole 2 a případně rozšíříme v kapitole 5.

Průvodce studiem

Studium této kapitoly vyžaduje ovládání elementárních dovedností při tvorbě informačních systémů a aplikací, včetně povědomí o životním cyklu softwarového produktu. Pro hlubší studium či detailnější zopakování jazyka UML si dovolím čtenáře odkázat na referenční a studijní literaturu na konci této kapitoly, protože jazyk UML je pouze doplňující nástroj vhodný pro zvládnutí a pochopení objektové orientace, a není tudíž hlavním tématem tohoto modulu o OOP.

Obsah

4.1	Co je to UML?	49
4.2	Modelování v UML	49
4.2.1	Stavební bloky	50
4.2.2	Diagramy tříd a objektů	51
4.2.3	Další diagramy UML	56
4.3	Závěr	60

Výklad

4.1 Co je to UML?

Jazyk UML (*Unified Modeling Language*) vznikl jako snaha sjednotit a standardizovat metodologie a metody objektově orientovaného návrhu a realizace aplikací přibližně v roce 1996 (Booch, Rumbaugh, Jacobson). V roce 1997 byl návrh UML přijat organizací OMG (Object Modeling Group, např. [28]) jako první průmyslový standard objektově orientovaného jazyka pro vizuální univerzální modelování. UML však není přímo metodologií návrhu systému, ale pouze prostředkem pro tvorbu modelů systému, a bývá často součástí nových metodologií.

Definice 4.1.1 *UML je jednotný grafický (vizuální) jazyk pro jednotnou specifikaci, vizualizaci, konstrukci a dokumentaci při objektově orientované analýze a návrhu (OOA&D) i pro modelování organizace (business modelling).* **Definice!**

UML umožňuje modelování softwarových aplikací stejně jako dalších systémů (např. obchodní procesy) jako kolekce spolupracujících entit (objektů). Základní cíl modelování je vizualizace, specifikace struktury a chování navrhovaného systému. Napomáhá dekomponování systému na dílčí části a zároveň nutí určit i jejich kompetence, vzájemné závislosti a interakci s okolím.

Úspěch tohoto způsobu modelování nejen objektově orientovaných systémů dokládají probíhající práce na nových verzích (koncem roku 2017 byla publikována specifikace UML verze 2.5.1).

4.2 Modelování v UML

Popis návrhu systému prostřednictvím jazyka UML se skládá většinou z několika pohledů. *Pohledem* (angl. *view*) nazýváme projekci systému (či jeho modelu) na jeden z jeho klíčových aspektů a znázorňujeme jej jedním či více diagramy UML. Základní pohledy jsou strukturální, datový, pohled na chování a rozhraní. Pohledy respektive diagramy lze rozdělit také na analytické („Co bude systém dělat?“) a návrhové („Jak to bude dělat?“). *Diagram* UML je struktura podobná obecnému diagramu grafu obsahující množinu grafických prvků (vrcholy) propojených vztahy (hrany).

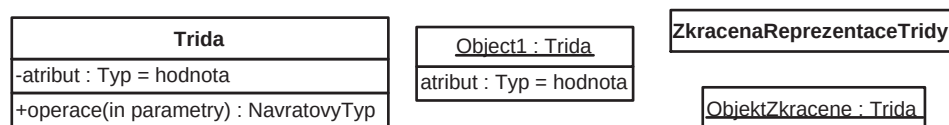
4.2.1 Stavební bloky

Pro přehlednost uvádíme krátký přehled základních stavebních bloků diagramů UML:

1. Prvky (neboli předměty, angl. *things*, či abstrakce, entity)
 - Strukturní: třída, případ použití, komponenta
 - Chování: interakce, stav
 - Seskupování: modul, balíček, podsystém (angl. *package*)
 - Doplňkové: komentáře a poznámky
2. Vztahy (angl. *relationships*) především strukturálních a sestavovacích prvků
 - Asociace (spojení mezi prvky)
 - Závislost (změna v jednom prvku ovlivní jiný závislý prvek)
 - Agregace, Kompozice (vyjádření dekompozice prvku na podčásti)
 - Generalizace (vztah mezi obecnějším a specifitějším prvkem)
 - Realizace (vztah mezi předpisem a jeho uskutečněním)

Z předchozích prvků potom mohu sestavit následující UML diagramy, které mohou být využívány při tvorbě různých UML pohledů:

- a) model struktury (statický aspekt modelu) - popisuje důležité entity systému a jejich souvislosti
 - Diagram tříd
 - Diagram komponent
 - Diagram rozmístění zdrojů (nasazení)
- b) model chování (dynamický aspekt modelu) - popisuje životní cyklus entit a jejich spolupráci
 - Diagram objektů
 - Diagram případů použití
 - Diagram interakce: diagram sekvence, diagram spolupráce
 - Stavový diagram
 - Diagram aktivit



Obrázek 4.1: *Symboły UML pro třídu, objekt a jejich zkrácené verze*

Některé diagramy mají spíše statický charakter (diagram tříd či komponent) a jiné jsou primárně určeny k zachycování dynamických vlastností popisovaného systému (diagramy interakce, stavové diagramy, diagramy aktivit).

Ve všech syntaktických konstrukcích UML je většina vlastností nepovinná, aby byla v tomto jazyce maximálně podpořena myšlenka efektivního modelování a prototypování při návrhu systému. Neuvedení těchto vlastností má samozřejmě vliv na kvalitu popisu modelu, ale nikoli na jeho validitu vůči syntaxi či sémantice UML.

4.2.2 Diagramy tříd a objektů

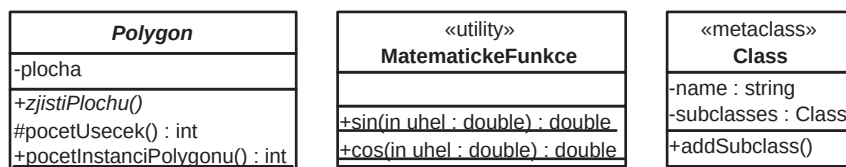
Nejpodrobnější popis si zaslouží prvky reprezentující třídu a objekt, ze kterých se potom pomocí vztahů komponují diagramy tříd a objektů. Proto si v následujícím oddíle popíšeme tyto dvě entity detailněji než další používané stavební bloky UML.

Reprezentace tříd a objektů v UML

Na obrázku 4.1 najdeme symboly reprezentující třídu, objekt a jejich zkrácené symboly. *Třída* je rozdělena na tři bloky obsahující název, třída seznam atributů a seznam operací. *Objekt* obsahuje pouze podtržený objekt identifikátor objektu s případným určením třídy a seznam atributů s přiřazenými hodnotami.

Atributy jsou prvky objektu (třídy), u nichž lze specifikovat typ atribut (obor hodnot), jméno (identifikátor), implicitní hodnotu a viditelnost. Při implementaci atributy často odpovídají instančním (třídním) proměnným nebo vlastnostem. Pro typy v UML zápisu se nutně nemusí využívat pouze datové typy konkrétního cílového implementačního jazyka. UML umožňuje u atributu specifikovat také režim možného přístupu pro čtení a zápis (např. jen pro čtení, jen pro zápis, pro čtení i zápis). Pro vyjádření atributů jen pro čtení se používá znak „/“ a nazývají se *odvozenými atributy*.

Operace ve většině případů odpovídají metodám třídy a jedná se o pojmenované chování třídy (potažmo instance třídy). Operace jsou popsány identifikátorem, modifikátorem viditelnosti, seznamem formálních parametrů a návratovým typem. Výjma identifikátoru jsou



Obrázek 4.2: Abstraktní třída a dvě ukázkové třídy se stereotypem

ostatní položky nepovinné. Každý parametr může obsahovat vstupně-výstupní modifikátor („in“ pro vstupní, „out“ pro výstupní a „inout“ pro vstupně-výstupní parametry), formální jméno a typ.

V UML najdeme tři typy viditelnosti atributů a operátorů (v závorkách je uvedena používaná prefixová značka):

soukromá viditelnost (-) atribut/operace je k dispozici pouze uvnitř třídy, kde je definován(a);

chráněná viditelnost (#) přístup je omezen pouze na instance tříd přímo či nepřímo zděděných od vlastníka atributu/operace;

veřejná viditelnost (+) k atributu/operaci lze přistupovat bez omezení z libovolného jiného objektu.

třídní atributy a operace

abstraktní operace a třída

Třídní atributy a operace jsou značeny podtržením (Neplést s podtržením identifikátoru v případě objektu!).

Abstraktní operace, u kterých nemá smysl mít v dané třídě konkrétní implementaci ale pouze definici, jsou značeny kurzívou. Stejně je tomu tak u abstraktní třídy na obrázku 4.2 (někdy je kvůli lepší přehlednosti vloženo do pole pro název třídy také klíčové slovo {abstract}. Připomeňme, že abstraktní třídy nelze instanciovat.

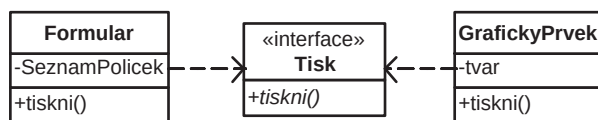
Stereotypy tříd

Vlastnosti třídy lze upřesnit pomocí tzv. stereotypu, což je označení speciálního druhu třídy se specifickým účelem. Stereotyp bývá zapisován nad název třídy do francouzských uvozovek (viz obr. 4.2).

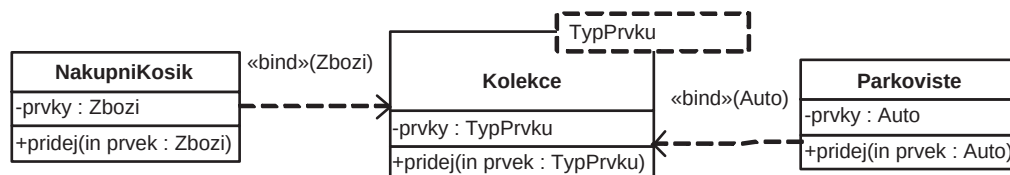
Často se v OOP setkáváme s třídami, které nejsou primárně určeny pro popis atributů a operací jejích instancí, ale pro sdružení obecně používaných funkcí (např. matematické). Takovýto význam třídy zapisujeme stereotypem «utility», neboli *balíček nástrojů*. V praxi se z této třídy nikdy netvoří instance (všechny členy má statické) nebo se vytváří pouze jediná instance (tzv. *singleton*, viz podkapitola 5.4.3). Někdy se využívá pro tvorbu *obálek* pro kód převzatý ze starších neobjektových jazyků (např. C nebo Pascal) a vede na tzv. objektově založené programování.

rozhraní

Stereotyp rozhraní slouží pro popis speciální třídy, která slouží pro sdružení množiny operací, jejichž rozhraní (deklarace operací,



Obrázek 4.3: Ukázka třídy se stereotypem rozhraní a jeho dvě realizace



Obrázek 4.4: Parametrizovaná třída Kolekce a dvě konkrétní třídy s navázaným parametrem na třídy Auto a Zbozi

protokol) vyváží ostatním třídám, které toto chování (rozhraní) pak povinně implementují (viz obrázek 4.3, uprostřed).

Méně používanými stereotypy jsou `«metaclass»`, `«type»`, `«implementation class»` a další.

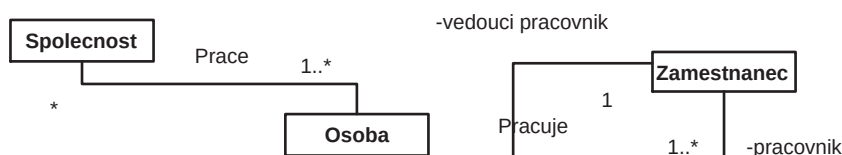
Parametrizovaná třída (generická třída, třída šablony)

Některé OOJ umožňují programovat s využitím parametrizovaných tříd, které uvnitř pracují s jedním nebo více v době psaní kódu neznámými datovými typy nebo třídami. Na obrázku 4.4 vidíme třídu šablony s jedním formálním parametrem zapsaným ve speciální kolonce v pravém horním rohu názvu třídy. Pokud provedeme navázání formálního parametru třídy na konkrétní datový typ či třídu, tak získáme novou třídu, kterou s původní parametrizovanou spojíme orientovanou úsečkou s návěštím `«bind»` a jménem dosazeného typu za požadovaný formální parametr parametrizované třídy.

Třída může dále obsahovat textové i grafické poznámky, klíčová slova (např. `abstract`), stereotypy a dodatečné informace pro dokumentační účely (např. jméno zodpovědného programátora, číslo poslední verze).

Diagram tříd

V předchozí sekci jsme si popsali, jak znázorňujeme třídy, objekty a jejich varianty. Nyní se tedy zaměříme na čtyři základní statické vztahy, které tyto entity propojují v diagramu tříd. Všimněme si také obecnosti těchto vztahů, které se neomezují pouze na výskyt v diagramech tříd, ale lze je uplatnit například i v diagramu případů použití a dalších.



Obrázek 4.5: Asociace s názvem a násobnostmi, pojmenovaná reflexivní asociace s rolemi

Definice!

Definice 4.2.1 Diagram tříd je graf symbolů tříd, rozhraní, seskupení a dalších strukturních prvků propojených statickými vztahy (asociace, závislost, agregace, kompozice, generalizace, realizace).

násobnosti vztahu

Násobnosti vztahu se udávají pro upřesnění obecného vztahu a bývají značeny na obou koncích úsečky (násobnosti označme a , b), která znázorňuje vztah mezi třídami A a B . Násobnost b v případě diagramu tříd odpovídá na otázku „S kolika objekty třídy B je v relaci jeden objekt třídy A ?“ a analogicky přesně naopak se dotazujeme na násobnost a . Násobnost se v UML zapisuje takto:

1. konstanta k (například 0, 1, 2, ...)
2. libovolné neomezeně velké nezáporné celé číslo (značíme $*$)
3. interval složený z konstanty určující dolní hranici D a horní hranici H , kdy platí $D < H$ nebo H může být $*$ (například $0..*$ pro libovolnou násobnost, $0..1$ pro volitelnost, apod.).

asociace

Asociace (angl. *association*) je obecná relace mezi dvěma či více třídami zakreslená úsečkou spojující tyto třídy. Asociace může být pojmenována pro upřesnění svého významu. Oba konce asociace mohou obsahovat popis role připojené třídy a násobnost vztahu na daném konci (obr. 4.5).

Atributy a operace se vztahem přímo k samotné asociaci sdružujeme do tzv. *asociační třídy*, která je znázorněna symbolem třídy propojené přerušovanou úsečkou s úsečkou vztahu (viz obrázek 4.6).

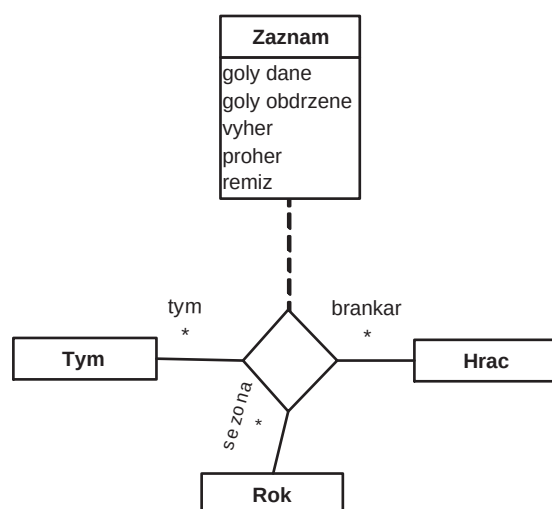
Na obrázku 4.6 vidíme také případ *vícenásobné asociace*, kdy se na vztahu podílejí více jak dvě třídy. Vztah je reprezentován prázdným kosočtvercem a úsečkami vedoucími do jednotlivých tříd vztahu.

realizace

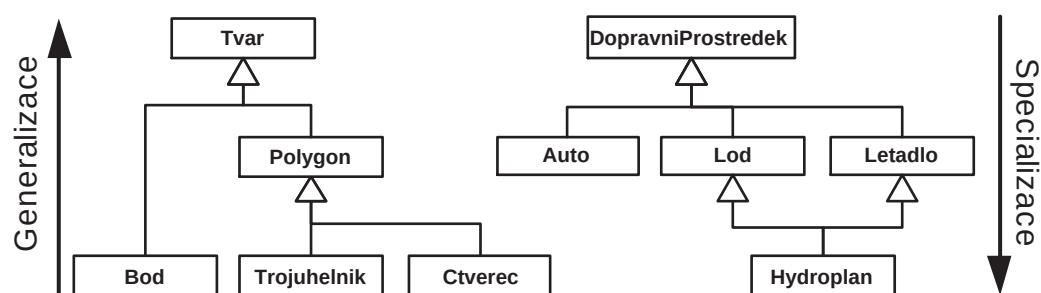
Realizace (angl. *realization*) je vztah mezi rozhraním a implementační třídou, který značíme čárkovanou šipkou směřující k rozhraní, jak to znázorňuje obrázek 4.3.

generalizace

Generalizace (angl. *generalization*) je statický vztah mezi obecnější a specifitější entitou, v případě tříd mezi rodičovskou třídou a třídou potomka. Jelikož je rodičovská třída obecnější než



Obrázek 4.6: Ukázka vícenásobné asociace i s asociační třídou



Obrázek 4.7: Jednoduchá a vícenásobná generalizace (dědičnost)

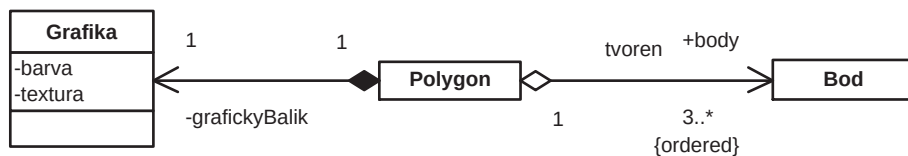
třídy potomků a šipka směřuje od potomka k rodiči, tak tento vztah nazýváme generalizace (zobecnění). Pokud bychom postupovali opačným směrem, půjde o specializaci, která se však v UML nepoužívá.

Popiskům generalizace říkáme *diskriminátory dědičnosti* a slouží pro upřesnění oblasti generalizace při dědění (např. u třídy *DopravniProstředek* a potomků *Auto* a *Kolo* bude diskriminátor dědičnosti *typPohonu*).

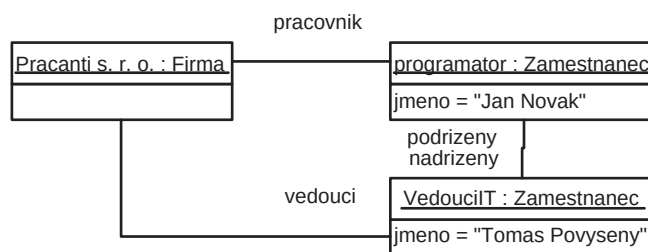
Grafický zápis neklade žádné omezení na modelování pomocí jednoduché či vícenásobné dědičnosti (viz obrázek 4.7).

Agregace vyjadřuje složení entity ze skupiny komponentních entit. Vztah kreslíme jako úsečku na straně celku zakončenou prázdným kosočtvercem (pevná násobnost je 1; neznačíme) a na druhém konci šipkou s případnou násobností.

Kompozice je speciální případ agregace, kdy každá komponentní třída smí náležet pouze jednomu celku (někdy tzv. silná agregace). Na rozdíl od agregace zakresluje na straně kompozitní třídy vyplněný



Obrázek 4.8: Agregáčnı́ a kompozitnı́ vztah



Obrázek 4.9: Ukázka diagramu objektů (instancı́)

kosočtverec.

Agregaci a kompozici lze někdy považovat za speciální případy asociačnı́ho vztahu. Příklad obou z nich je na obrázku 4.8.

závislost

Závislost (angl. *dependency*) je vztah mezi prvky, v nı́ž změna jednoho elementu (dodavatele) má vliv na závislý element (klienta). Využívá se především pro vyjádření závislostı́, jež nejsou asociací a značí se přerušovanou šipkou od klienta k dodavateli. Navíc může obsahovat upřesnění formou stereotypu, např. «use», «friend», «bind» (obr. 4.4) nebo mezi třídou a její instancı́ «instantiate».

4.2.3 Další diagramy UML

Další diagramy UML si popı́šeme jı́ž pouze stručně, případně na modelovém příkladu, kde se budeme snažit zachytit nejčastější a nejpoužívanější notace specifické pro daný diagram.

Diagram objektů

Diagram objektů (instancı́) ukazuje objekty a jejich propojení, včetně identifikace význačných objektů v určitém čase. Hlavním účelem tohoto pohledu bývá pomoc při pochopenı́ složitějších vazeb mezi třídami (v diagramu tříd) prostřednictvım ilustrace na konkrétnım příkladě několika instancı́ těchto tříd (obrázek 4.9). Protože reprezentují také statickou strukturu v konkrétnım časovém okamžiku, lze diagramy objektů používat i pro ověření korektnosti diagramu tříd, za jejíž instanci lze diagram objektů považovat.

Diagramy seskupení, komponent a zdrojů

Následující tři typy diagramů jsou většinou nově zavedeny či upraveny v UML verze 2 a jejich detailnější popis lze najít ve studijní literatuře (viz konec této kapitoly).

Diagram seskupení je vhodný především pro modelování rozsáhlých systémů a prezentaci vazeb mezi jeho podsystémy a dalšími moduly. *Seskupení* (angl. *package*) je prvek diagramu, který dokáže obalit a zapouzdřit libovolnou množinu entit, pro kterou to dává významový smysl, za účelem zpřehlednění vazeb mezi těmito množinami. Seskupení je značeno obdélníkem (s „ouškem“ pro popisek), který obepíná všechny zahrnované entity. Diagram seskupení je statického charakteru, protože reprezentuje seskupování v čase kompilace (překladu).

Diagram komponent ukazuje strukturu komponent systému a závislosti mezi nimi. *Komponenta* je přístupná pouze prostřednictvím svého rozhraní a je dána svými klasifikátory (specifikují komponentu) a artefakty (implementují komponentu). Tento typ diagramu je již velmi blízký samotné implementaci a bývá často využíván pro dokumentaci nebo odhalení správného pořadí kompilace komponent systému a zároveň tříd, které tuto komponentu implementují.

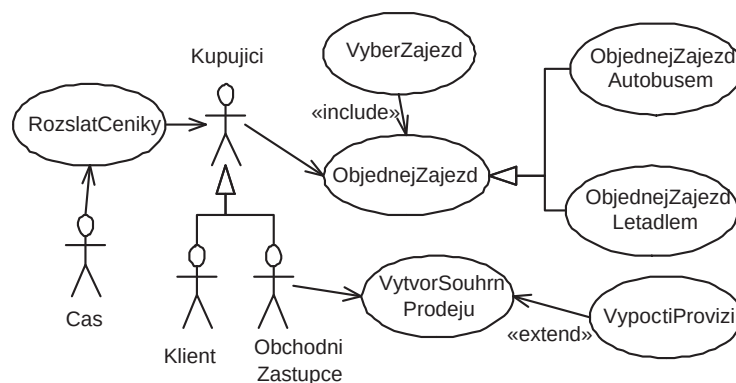
Diagram rozmístění zdrojů poskytuje pohled na fyzické rozmístění systému, který je v tomto diagramu reprezentován pomocí uzlů propojených komunikačními cestami. *Uzel* je v tomto grafu výpočetním zdrojem rozděleným na *procesor* (schopný spouštět komponenty) a *zařízení* (neumí spouštět komponenty).

Diagram případů použití

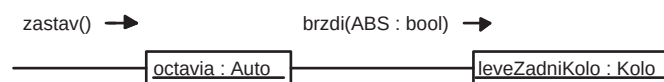
Při tvorbě specifikace a analýzy požadavků aplikace máme kromě slovního popisu detailů případů použití k dispozici také diagram případů použití, který graficky znázorňuje účastníky, případy použití, interakce a hranice systému.

Na obrázku 4.10 vidíme několik případů použití (ovály s popiskem uvnitř), které jsou využívány aktéry (Kupující, Čas). Entity aktérů i samotných případů použití lze generalizovat pro zvýšení přehlednosti diagramu podobně jako v případě diagramu tříd (např. v případě aktérů Klient a ObchodniZastupce). Pomocí stereotypů «use» (implicitní), «include» a «extend» lze upřesnit vztahy mezi samotnými případy použití.

Diagram datových toků (angl. *Data Flow Diagram*) je hierarchický model používaný při strukturované analýze a návrhu pro specifikaci chování systému. Na rozdíl od objektově orientovaného diagramu případů použití je blíže návrhu, protože obsahuje i informace o datech.



Obrázek 4.10: Diagram případu použití



Obrázek 4.11: Diagram spolupráce mezi dvěma jednoduchými objekty

Diagramy interakce

Základem vykonávání objektově orientovaného programu je zasílání zpráv. Zpráva je požadavek odesílajícího objektu o vykonání nějaké operace v cílovém objektu (tzv. příjemce zprávy).

Interakční diagramy objektů popisují komunikační strukturu v době běhu modelovaného systému (tzv. dynamický popis) a jsou reprezentovány diagramem spolupráce (kolaborace) a diagramem sekvence.

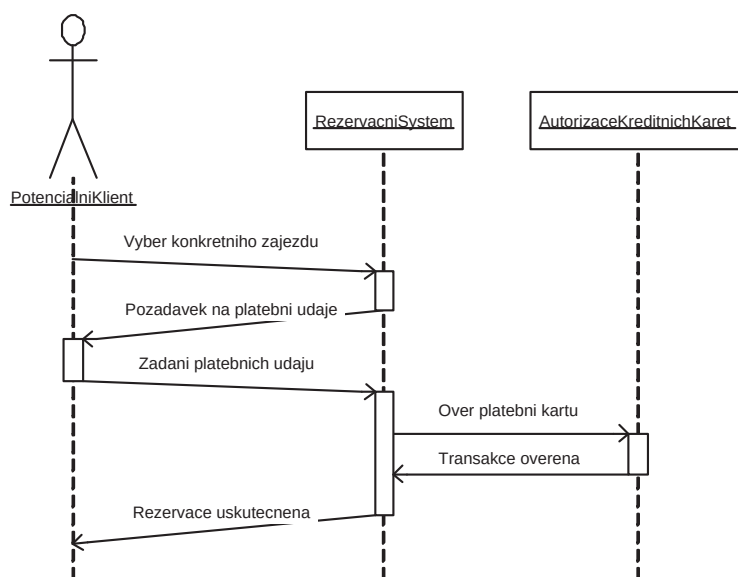
diagram spolupráce

Diagram spolupráce popisuje objekty (instance tříd ze statických pohledů) propojené pomocí posílání zpráv mezi těmito objekty (včetně jmen zpráv a jejich parametrů; viz obrázek 4.11, šipka směřuje od odesílatele k příjemci).

diagram sekvence

Druhým významným popisem dynamických vztahů v programu je *diagram sekvence*, který zdůrazňuje časovou posloupnost vztahů mezi objekty. V některých případech jej lze využít pro detailnější popis (upřesnění) vybraného případu použití.

Příklad diagramu sekvence na obrázku 4.12 obsahuje tři objekty, z nichž vede směrem dolů úsečka (směr toku času) reprezentující linku života objektu. Aktivitu/převzetí toku řízení naznačujeme zesílením linky na podlouhlý obdélník. Šipky s návěštími mezi jednotlivými linkami života znázorňují synchronní či asynchronní komunikaci mezi příslušnými objekty.



Obrázek 4.12: Diagram sekvence zjednodušeného rezervačního systému a ověření kreditních údajů



Obrázek 4.13: Stavový diagram článku jednoduchého redakčního systému

Stavový diagram

Stavový diagram (obrázek 4.13) se váže vždy na konkrétní třídu a ukazuje stavy, do kterých mohou její instance vstupovat, a přechody mezi nimi. Při pokročilejším modelování tyto diagramy umožňují popisovat vnořené stavy (modularita), používané zprávy i s argumenty, souběžnost a synchronizaci.

Úlohy k procvičení:

Pokuste se namodelovat pomocí jazyka UML vybraný diagram samotného UML. Jedná se o konstrukci tzv. metamodelu, kdy se snažíme popsat jazyk pomocí něj samotného. Tato vlastnost u libovolného jazyka zpravidla naznačuje jeho univerzálnost a flexibilitu.

Klíč k řešení úloh

Inspirace pro řešení naleznete v knize [2].

4.3 Závěr

Tato kapitola zdaleka nepopsala všechny možnosti a syntaktické obraty jazyka UML, ale naznačila jeho hlavní aspekty a modelovací prvky potřebné pro elementární používání při objektově orientované analýze, návrhu i programování.

Více informací najdete jak v mnoha knihách (i v češtině) specializovaných na tento jazyk, tak například ve studijní opoře k předmětu IUS, kde je tento jazyk více provázán také s výkladem vybrané metodologie. Najdete zde také množství tipů a jednoduchých příkladů pro konstrukci objektově orientovaných modelů pomocí UML.

Závěr

UML je jednotný vizuální jazyk pro specifikaci, dokumentaci a modelování nejen při objektově orientované analýze a návrhu.

UML diagramy jsou tvořeny stavebními bloky (prvky a vztahy mezi nimi). Podle aspektu modelování pak diagramy dělíme na statické (modely struktury) a dynamické (modely chování).

Hlavní pozornost je věnována diagramům objektů, tříd a vztahů mezi nimi. V závěru jsou stručně probrány i ostatní diagramy.

Další zdroje

K tomuto tématu je i na českém knižním trhu k dispozici velké množství kvalitních publikací (většinou překlady z anglických originálů). Prohloubení znalostí se zaměřením na vztah UML k objektově orientovaným jazykům naleznete v knize:

- Jones, M. P.: Základy objektově orientovaného návrhu v UML, Grada Publishing s.r.o., 2001, (překlad do češtiny Karel Voráček, Fundamentals of Object-Oriented Design in UML; Addison-Wesley, 2000), ISBN 80-247-0210-X.

Ještě přehledněji a srozumitelněji je napsána kniha s obrovským množstvím příkladů provázaných s celým procesem vývoje aplikací:

- Arlow, J., Neustadt, I.: UML a unifikovaný proces vývoje aplikací, Computer Press, Brno, 2003 (Addison-Wesley, 2002), ISBN 80-7226-947-X.

Poslední dostupné texty vhodné pro rozšíření této kapitoly jsou:

- Kočí, R., Křena, S.: Studijní opora předmětu Úvod do softwarového inženýrství (IUS), FIT VUT Brno, 2006.
- Objektově orientované modelování systémů, učební text zaměřený na jazyk UML 2.0, FIT VUT Brno, 2004.

Dostupné na stránkách předmětu Úvod do softwarového inženýrství (IUS) (únor 2006).

Kapitola 5

Vlastnosti objektově orientovaných jazyků

Tato kapitola se podrobněji zabývá pokročilejšími vlastnostmi především třídních jazyků založených na objektově orientovaném paradigmatu. Jsou však také diskutovány moderní techniky a vlastnosti programovacích jazyků, které jsou sice nejčastěji svázány s objektovým paradigmatem, ale nejsou na něj bezpodmínečně vázány. V druhé části se diskutuje otázka zpracování a překladu zdrojových textů objektově orientovaných jazyků.

Čas potřebný ke studiu: 5 hodin.

Cíle kapitoly

Cílem této kapitoly je poskytnout čtenáři povědomí i o pokročilejších technikách a principech používaných jak při programování v objektově orientovaných jazycích, tak při implementaci jejich překladače nebo interpretu.

Průvodce studiem

Od studenta se předpokládá aktivní znalost konceptů objektové orientace i všech důležitých pojmů z kapitoly 2. Dále požadujeme i povědomí o principech konstrukce překladačů a interpretů jazyků nižších než jsou objektově orientované (především modulárních).

Obsah

5.1	Úvod	63
5.2	Poznámka ke klasifikaci jazyků	63
5.2.1	Pojmy	63
5.2.2	Klasifikace jazyků	64
5.3	Vlastnosti třídních jazyků	65
5.3.1	Řízení toku programu	66
5.3.2	Jmenné prostory	66
5.3.3	Modifikátory viditelnosti	66
5.3.4	Přetěžování metod	67
5.3.5	Vícenásobná dědičnost	67
5.3.6	Rozhraní	69
5.3.7	Výjimky	71
5.3.8	Šablony	72
5.3.9	Systémy s rolemi	74
5.4	Poznámky k implementaci OOJ	76
5.4.1	Manipulace se třídami	76
5.4.2	Virtuální stroj	78
5.4.3	Poznámka o návrhových vzorech	80
5.5	Zpracování - analýza, vyhodnocení, interpretace, překlad	81
5.5.1	Překladač	81
5.5.2	Interpret	82
5.6	Závěr	83

Výklad

5.1 Úvod

Na úvod kapitoly si definujeme obecnější pojmy nejen z oblasti programovacích jazyků, které budeme v textu používat.

Ortogonalita je obecně řečeno nezávislost jisté technologie na kontextu použití a lze jí tedy využít i v jiném kontextu.

Definice 5.1.1 *Konkrétně v textu o programovacích jazycích chápeme ortogonalitu jako nezávislost vlastnosti jazyka na programovacím paradigmatu.* **Definice!**

Například šablony a výjimky jako technologie jsou ortogonální vůči objektově orientovanému paradigmatu, z čehož plyne, že je lze uplatnit i v jazycích, jež nejsou objektově orientované.

5.2 Poznámka ke klasifikaci jazyků

V kontextu s OO jazyky se objevují některé nové nebo se více diskutují dříve méně časté typy jazyků. Nyní stručně shrneme a upřesníme následující kategorizaci (zdroj od zdroje se mírně liší). Vycházíme především ze striktnějšího přístupu k následujícím vlastnostem jazyků (viz [31]). Klasifikace jazyků podle několika významných kritérií (práce s typy): kritéria

1. podle určování typů při zápisu programu;
2. podle doby vytvoření vazby proměnné na typ;
3. podle způsobu typové kontroly;
4. podle důkladnosti typové kontroly.

Nejprve ještě projdeme několik základních pojmů, které budeme dále potřebovat.

5.2.1 Pojmy

Datový typ popisuje reprezentaci, interpretaci a především strukturu hodnot používaných algoritmy nebo objekty v paměti počítače. *Typový systém* pak typové informace využívá k ověření korektnosti počítačových programů při jejich práci s daty.

Praktičtěji řečeno je typ většinou pojmenovaná doména možných hodnot a vlastností/chování konkrétní entity jazyka (proměnné, funkce, metody, objektu).

typová chyba

Typová chyba je ohlášení chyby (při překladu nebo i za běhu), která označuje použití nepovolených/nekompatibilních/nevhodných typů operandů při libovolné operaci.

silná kontrola

Silně typová kontrola je takový typ kontroly, kdy jsou vždy detekovány všechny typové chyby, což vyžaduje schopnost určit typy všech operandů (proměnných či výrazů v případě operací; parametrů v případě funkcí či metod), ať již při kompilaci (staticky typované jazyky) nebo za běhu programu (dynamicky typované jazyky).

Nyní je popíšeme podrobněji spolu s příklady nejen objektově orientovaných jazyků.

5.2.2 Klasifikace jazyků

Podle určování typů při zápisu programu:

beztypové v podstatě se jedná pouze o teoretické jazyky a formalismy, ve kterých nemá u proměnných smysl uvažovat typ, protože tyto jazyky s hodnotami v proměnných nijak nepracují (např. ζ -kalkul a λ -kalkul).

netypované proměnná nemá určen ve zdrojovém kódu typ a tuto informaci nemá programátor k dispozici (resp. jen pomocí explicitních dotazovacích funkcí na typ, pokud je vůbec jazyk/knihovna jazyka nabízí); interpret/kompilátor zajišťuje implicitní typové konverze automaticky. Např. BASIC, shell, PHP, Python.

typované typ je určen ve zdrojovém programu u každé entity. Proměnná v těchto jazycích může mít typ určen *explicitně* (provázáním typu a proměnné na úrovni zdrojového programu) nebo *odvozením* (tzv. *typová inference*), kdy je výsledný typ výrazu odvozen z typů jeho operandů a samotných operací (vyhodnocením typového výrazu nebo pomocí daných odvozovacích pravidel).

Podle doby vytvoření vazby proměnné na typ:

staticky typované před během programu, tj. v době návrhu kompilátoru, kompilace programu či zavádění programu; např. Delphi, C++, Java. Typování proměnné může být buď implicitní (např. podle jména proměnné), nebo explicitní (nejčastější).

dynamicky typované typ proměnné je určen až za běhu programu; např. jazyk Smalltalk nebo Python. Všechny proměnné obsahují objekty, ale konkrétní typ záleží až na konkrétních přiřazených instancích (nejčastěji je typ závislý na třídě dané instance, ale třída nutně nemusí přesně korespondovat s typem, protože podstatný je skutečný typ objektu).

Podle způsobu typové kontroly:

statická typová kontrola kdy je většina typových kontrol prováděna během překladu (stále však existují i u těchto jazyků kontroly prováděné dynamicky za běhu; např. variantní záznamy v C/C++, jazyce Ada nebo Pascalu)

dynamická typová kontrola veškeré typové kontroly mohou být prováděny až za běhu programu, což však ovlivňuje i výkon programů v těchto jazycích a často vyžadují běh na virtuálním stroji. Například jazyk Smalltalk, JavaScript, Python nebo SELF.

Podle důkladnosti typové kontroly:

Toto kritérium kategorizuje především typované jazyky se statickou typovou kontrolou:

- *silně typované jazyky* jsou jazyky, kde neexistuje možnost, jak implicitně způsobit typovou chybu/nekonzistenci s výjimkou explicitní typové konverze s vypnutou typovou kontrolou (např. Modula-3, Java nebo Ada). U některých jazyků dokonce nejsou možné žádné implicitní konverze (některá varianta ML nebo některé implementace Smalltalku či jazyka SELF, což jsou jazyky dynamicky typované).
- *slabě typované jazyky* jsou jazyky (např. Pascal, Fortran), kde je možné i po úspěšné kompilaci dojít k typové chybě (např. variantní záznamy v Pascalu); například C/C++ umožňují kromě volnější práce s variantním typem (union) nekontrolovat typy parametrů funkcí/metod a další typové prohřešky.

5.3 Vlastnosti třídních jazyků

Tato sekce se podrobněji věnuje některým specifitějším vlastnostem třídních jazyků. Nejprve je nastíněno řízení toku programu v hybridně objektově orientovaných jazycích a poté je zmíněno několik důležitých vlastností moderních OOJ, jež vhodně doplňují znalosti z kapitoly 2.

5.3.1 Řízení toku programu

Jak jsme si již uvedli, u hybridních OOJ se zachovávají vedle objektových i klasické primitivní a strukturované typy. Protože ale tyto staré typy nerozumí zprávám, je nutno vedle mechanismu zasílání zpráv zachovat i staré konstrukce ze strukturovaných a modulárních jazyků jako příkazy (větvení, cykly, atd.), výrazy a klíčová slova (např. označení prim. typů, řídicí příkazy). Výhodou těchto jazyků je pozvolnější přechod z modulárních jazyků na objektově orientované (uživatel není nucen hned začít myslet plně objektově), ovšem s rizikem, že se může uživatel naučit špatnému objektovému myšlení. Pro podporu pochopení objektového paradigmatu je proto lepší si alespoň vyzkoušet některý čistě objektový jazyk. Další nevýhodou tohoto přístupu je komplikovanější syntaxe i sémantika (například různý způsob práce s proměnnými potažmo hodnotami, jež jsou buď primitivního typu, nebo objektového).

5.3.2 Jmenné prostory

Jmenný prostor (angl. *namespace*) je z programátorského hlediska kontejner pro identifikátory (či jiné entity jazyka). V tomto prostoru platí pravidlo, že uvnitř se nevyskytují žádné dva stejné identifikátory (pokud bereme v úvahu jejich plnou kvalifikaci). V UML lze jmenný prostor modelovat pomocí entity seskupení. Důvodem zavádění jmenových prostorů do jazyků je zabránění kolizí jmen v rozsáhlých zdrojových textech, kde se běžně využívají množiny tříd více autorů. Tato jazyková vlastnost nemá, podobně jako přetěžování metod a operátorů, nezbytnou vazbu na objektově orientované jazyky, ale je v nich s výhodou velmi často využívána.

5.3.3 Modifikátory viditelnosti

Modifikátor viditelnosti je mechanismus přítomný v mnoha dnešních OOJ, který má za úkol měnit možnost přístupu k různým entitám jazyka, a tak podrobněji konfigurovat koncept zapouzdření. Takto ovlivňovanými entitami bývají nejčastěji atributy a metody, dále vlastnosti a někdy i dědičnost (pro systematické omezení viditelnosti všech entit v nové třídě). Již v jazyce UML (viz kapitola 4) jsme se mohli setkat se třemi základními modifikátory:

- soukromý (angl. *private*) — atributy či metody jsou přístupné pouze z metod stejné třídy
- chráněný (angl. *protected*) — atributy či metody jsou dostupné pouze z metod stejné třídy a zděděných podtříd

- veřejný (angl. *public*) — atributy či metody jsou dostupné odkudkoliv (z libovolné metody)

V některých jazycích k nim mohou přibýt i další specifitější modifikátory. Například v případě jazyka se jmennými prostory má smysl uvažovat modifikátor, který zajistí veřejnou viditelnost pouze uvnitř domovského jmenného prostoru (např. `internal` v C#). Za účelem efektivnější implementace může být výhodné mít možnost přistupovat k soukromým entitám v případě tzv. spřátelených tříd, které jsou v blízkém závislostním vztahu (např. `friend` v C++).

Jiné jazyky zase považují porušování zapouzdření za zavrženíhodný nešvar a napevno viditelnost určují (např. soukromá viditelnost atributů a veřejná viditelnost všech metod v jazyce Smalltalk).

5.3.4 Přetěžování metod

Přetěžování metod (angl. *method overloading*) je vlastnost (opět není doménou pouze objektově orientovaných jazyků) umožňující definovat ve třídě (respektive v protokolu objektu) více metod se stejným jménem. Jediným požadavkem bývá, aby se metody odlišovaly v typech nebo v počtu parametrů.

Kromě přetěžování metod se někdy umožňuje přetěžovat i operátory (unární i binární), což ovšem nemusí být za všech okolností vhodné.

Úlohy k procvičení:

Zamyslete se nad tím, proč nestačí, aby se signatura přetěžovaných metod lišila pouze v návratovém typu. Dále najděte příklad jazyka, který podporuje přetěžování funkcí a není objektově orientovaný.

5.3.5 Vícenásobná dědičnost

V definici přímých předků dané třídy může být uvedena i více jak jedna třída. Typickým reprezentantem je jazyk C++ nebo Python, dále třeba CLOS. Tento typ dědičnosti má však bohužel více nevýhod než výhod, které v následující sekci probereme detailněji:

- opravdu oprávněné a správné použití je velmi ojedinělé (často pouze svádí k chybám v návrhu);
- efektivní implementace patří mezi pokročilejší problémy návrhu OOJ.

Příklad 5.3.1 *Vícenásobná dědičnost, demonstrace konfliktů jmen.*

```

class A is
    int d;
    float dA;
    float f(int);
endOfClass

class B is
    int d;
    float dB;
    float f(int);
    int g(float);
endOfClass

class C inherited
    from A, B is
        int dC;
        int h(int);
endOfClass

```

problémy

V příkladu jsou viditelné následující problémy, které musí překladač OOJ s vícenásobnou dědičností řešit:

1. rodičovské třídy mohou obsahovat členy stejného jména i typu;
2. nebo jen stejného jména a různého typu;
3. pořadí volání konstruktorů (tzv. pořadí inicializace), případně destruktory;
4. uložení objektu třídy *C* v paměti tak, aby jej bylo možno využít kdekoli je očekáván objekt třídy *A*, *B* nebo *C* (např. v rámci vyžadované dědičnosti).

návrhy řešení

Návrhy řešení některých problémů vícenásobné dědičnosti:

ad 1) Kolize metod stejného jména i typu v nové třídě lze řešit třemi způsoby: (a) sémantickými kontrolami zakázat tuto možnost již při kompilaci zdrojového kódu, (b) dožadovat se v případě kolize upřesnění, ve které třídě volanou metodu hledat (tzv. kvalifikace metody) a nebo (c) vyžadovat povinnou reimplementaci, což má ovšem svá úskalí (např. při využití na místě, kde je očekáván objekt rodičovské třídy).

ad 2) Kolize metod se stejným jménem ale různým návratovým typem u nové třídy, kde se buď (a) tento případ zcela zakáže podobně jako v předchozím případě, nebo (b) se povolí existence obou metod ovšem pouze v případě, že daný jazyk umožňuje přetěžování, což kromě překladu zvyšuje i režii u pomocných rutin za běhu programu.

ad 1 a 2) Atributy stejného jména i typu jsou řešeny analogicky k problému s metodami. Pouze v případě stejně pojmenovaných atributů jiných typů se zatím nepodařilo úspěšně implementovat přetěžování, takže se tato kombinace zpravidla zakazuje.

ad 3) Problém s inicializací se týká pořadí volání konstruktorů (metody sloužící pro inicializaci objektu při jeho vytváření), jež jsou automaticky spouštěny nejen z třídy daného objektu, ale většinou i z rodičovských tříd. První možností je využití pořadí zápisu tříd ve zdrojovém textu, v jakém byly v seznamu rodičovských tříd instanciovány uvedeny. Toto řešení většinou využívá algoritmu prohledávání do hloubky v hierarchii dědičnosti od dané třídy směrem k předkům a vrací první nalezenou shodu, čímž se stává deterministickým i v případě více cest k některému z předků. Přesnější stanovení pořadí i algoritmus hledání jsou specifické pro jednotlivé OOJ s vícenásobnou dědičností. Dalším způsobem je ignorování zápisu ve zdrojovém textu, ale využití uživatelem definovaného pořadí inicializace, které ovšem není dodatečně kontrolováno na případné uvážnutí (angl. *deadlock*) či nesprávné pořadí.

ad 4) Ani pro způsob ukládání objektů v OOJ s vícenásobnou dědičností neexistuje jedno univerzální a obecné řešení, které navíc většinou silně závisí na vlastnostech daného jazyka. Typicky lze ukládat členy jednotlivých tříd v objektu do bloků podle tříd a v případě kolize jmen vytvořit duplikát daného člena v obou příslušných blocích. Při očekávání rodičovské struktury u takového objektu je však třeba zajistit synchronizaci obsahu duplikovaných atributů.

5.3.6 Rozhraní

Výhodou vícenásobné dědičnosti oproti jednoduché je možnost sdílení motivace protokolu napříč hierarchií dědičnosti, tj. i u tříd z různých větví stromu dědičnosti lze syntakticky zaručit existenci patřičného podprotokolu. Hlavní nevýhodou je však problém s kolizí různých implementací pro stejně pojmenované metody (případně i atributy). Zachováním popsané výhody a vyloučením nevýhod dostáváme koncept *rozhraní* (angl. *interface*), který je schopen s jistými omezeními nahradit vícenásobnou dědičnost.

Přestože je koncept rozhraní znám již delší dobu a jeho implementace je mnohem jednodušší než u vícenásobné dědičnosti, tak se stává populární až v moderních OOJ s jednoduchou třídní dědičností jako Java a C#. Rozhraní však není omezeno pouze na objektově orientované jazyky (např. funkcionální jazyk Haskell).

Definice 5.3.1 *Rozhraní je schéma, které deklaruje seznam metod (jména, parametry, návratové typy) a případně i několik nadrozhraní. Použití rozhraní na jistou třídu pak vynucuje implementaci všech metod uvedených v rozhraní a ve všech jeho nadrozhráních. I v pří-* **Definice!**

padě jednoduché dědičnosti lze třídu zavázat k implementaci několika rozhraní.

Zjednodušeně lze na rozhraní pohlížet jako na speciální třídu, která obsahuje pouze abstraktní metody (tj. metody bez implementace) a zakazující definici instančních proměnných. Koncept rozhraní lze využívat i v ostatních programovacích paradigmatech.

Příklad 5.3.2 *Definice a použití rozhraní.*

```
interface Comparable defines
    bool isEqual(class Comparable, class Comparable);
endOfInterface
interface Orderable extends Comparable defines
    bool lessThan(class Orderable, class Orderable);
endOfInterface
interface Printable defines
    void print(class Printable);
endOfInterface

class IntegerNumber implementing Orderable, Printable is
    ...
    bool isEqual(class Comparable, class Comparable);
    bool lessThan(class Orderable, class Orderable);
    void print(class Printable);
    ...
endOfClass
class Square implementing Printable is
    ...
    void print(class Printable);
    ...
endOfClass
```

Nyní můžeme vytvořit funkci, jež by řadila pole libovolných objektů, u nichž by byl jediný požadavek—jejich třída musí implementovat rozhraní `Orderable`.

```
sort(class Orderable[])
```

Koncept rozhraní tedy podporuje tzv. generický polymorfismus. To je odlišný přístup než například v případě tradičního přetěžování operátorů, kdy operátor definujeme pro každý typ zvlášť. Naše generické řešení tak bude použitelné i pro typy/třídy/objekty, které ještě v době implementace tohoto řešení neexistovaly.

implementace

I když je implementace rozhraní jednodušší než v případě vícenásobné dědičnosti, přesto je v případě kombinace s jednoduchou dědičností nutno zvolit nejvhodnější implementaci:

- a) vázání přes jméno rozhraní (nikoliv přes adresu; nutno provádět za běhu a často s podporou virtuálního stroje; např. Java, která obsahuje v bytekódu (angl. *bytecode*) plnou jmennou kvalifikaci rozhraní);

- b) několik tabulek virtuálních metod (pořadí je dáno zápisem rozhraní ve zdrojovém textu);
- c) metody `get/set` (vázání přes jméno pro kompilované jazyky; metody `get` a `set` slouží pro získání adres kódu metody zadaného jména).

Nevýhoda oproti vícenásobné dědičnosti—nelze zajistit sdílení implementace jistého chování napříč hierarchií dědičnosti, což lze ovšem obcházet pomocí pokročilých návrhových vzorů (viz [11]).

Otázka

Mějme rozhraní `Comparable` z předchozího příkladu, které vyžaduje implementaci metody (zprávy) `isEqual`, tzv. požadovaná metoda. Uvažujme ale případ, kdy chceme sdílet v rámci tohoto rozhraní i tzv. odvozené metody, jejichž výpočet je založen na požadovaných metodách, ale jinak je univerzálně platný. Například zpráva `isNotEqual`, jež lze implementovat pouhou logickou negací výsledku metody `isEqual`. Nyní ovšem nastává problém, protože definice rozhraní neumožňuje, aby rozhraní obsahovalo implementaci. Pokuste se navrhnout funkční řešení tohoto problému.

Klíč k řešení úloh

Prostudujte v [11] vhodné návrhové vzory pro vyřešení tohoto problému.

5.3.7 Výjimky

Moderní způsob ošetřování chybových a neočekávaných stavů vykonávaného programu je využití výjimek (angl. *Exception*), což je mechanismus, který popisuje způsob šíření informace o chybě, způsob zastavení/přeskočení výpočtu a umožňuje provést ošetření chyby až za samotným algoritmem a tím zlepšit čitelnost kódu i samotného algoritmu. V neposlední řadě umožňuje také spouštění finalizační sekce, jejíž provedení je garantováno i v případě vyvolání výjimky (tj. deklarativně označená sekce, která nesmí být vynechána při přeskakování chybné nebo chybou ovlivněné části programu).

Příklad 5.3.3 *Nejčastější programová struktura try-catch-finally pro práci s výjimkami nejen v OOI.*

```
class FileException inherits from Exception is
...                               // definice nové třídy výjimek
endOfClass
...
```

```

void g()
{
    if ((fIn = fopen(file, "r")) == NULL)
    {
        e = FileException(file, "r");    // vytvoření objektu výjimky
        e.where("module.fun");           // inicializace výjimky
        rise(e);                         // vyvolá výjimku
    }
    endif
end;
...
try
{
    ... g(); ...                        // blok sledovaný na výjimky
}
catch(e)
{
    if (e is FileException)
    {
        ...                            // zpracování/ošetření výjimky
    }
}
finally
{
    ...                                // nevynechatelný kód, finalizační sekce
}
endTryCatchFinally

```

Při vyvolání výjimky uvnitř *try*-bloku (příkazem či zprávou *rise* nebo v některých jazycích příkazem či zprávou *throw*) je proveden skok na začátek *catch*-bloku a na jeho formální parametr je navázán vzniklý objekt výjimky. V ošetřovacím kódu (*catch*-blok) je pak většinou potřeba zjistit, jaké třídy výjimka je, abychom věděli, jak s ní naložit.¹

Jak již bylo zmíněno v úplném úvodu kapitoly, tak výjimky jsou ortogonální technikou vůči OOP, kterou lze využívat i ve strukturovaných/modulárních či deklarativních jazycích (např. Haskell).

5.3.8 Šablony

Šablony mohou být implementovány v zásadě třemi způsoby:

1. staticky (šablona je zpracovávána a využívána pouze při kompilaci zdrojového kódu; v podstatě se jedná o velmi sofistikované rozšíření preprocesoru jazyka, např. maďarská notace v C++)
2. dynamicky (v případě hybridních jazyků nejčastěji pomocí virtuálních funkcí resp. metod, což vyžaduje dodatečnou režii za běhu programu)
3. ad hoc (v jazyku předem a napevno definované přetěžováním operátorů pro omezený počet typů, např. Pascal a jeho operátor $+$ pro sčítání číselných typů i konkatenaci řetězců)

Příklad 5.3.4 Ve staticky typovaných jazycích často potřebujeme budovat homogenní strukturovaný datový typ. Problémem je, pokud danou strukturu potřebujeme pro předem neznámý výčet typů jeho prvků nebo je tento výčet nemalý. Bez šablon bychom museli provést

¹Ke zjištění typu výjimky lze samozřejmě kromě reflexe využít i uživatelské členy struktury, která výjimku reprezentuje.

definici strukturovaného typu pro každý typ jeho prvků zvlášť (a to nemluvě o typech, které teprve vzniknou někdy v budoucnu).

Cvičení: Navrhněte třídu reprezentující abstraktní datový typ `BinaryTree` pro binární strom. Uvažujte požadavek na homogenitu jeho prvků. Jakým způsobem lze zadání implementovat v OOI bez šablon a se šablonami?

Definice 5.3.2 *Šablona je mechanismus, který umožňuje parametrizaci definic datových typů (a potažmo i tříd). V definici nového šablonového typu potom daný parametr využíváme jako proměnnou, která obsahuje identifikaci jiného typu. Šablony mohou obsahovat i více parametrů.* **Definice!**

Šablonový typ je nutné před použitím ve zdrojovém textu instanciovat. Tj. dosadit za parametry konkrétní datové typy. Až z konkretizovaného šablonového typu lze vytvářet proměnné daného typu. Výhoda šablonového typu je, že může být využit i jako parametr funkcí a metod a tím zvýšit flexibilitu těchto funkcí v době překladu (nikoli však v době běhu, kdy již jazyky vyžadující šablony potřebují mít ve všech typech jasno).

Cvičení - pokračování: *Objektově orientované řešení bez šablon ve staticky typovaných jazycích*

Pro uzly bychom vytvořili abstraktní třídu `TreeNode`, která by obalovala prvky stromu. Třída pro samotný strom `Tree` by pak k manipulaci s těmito uzly využívala abstraktní operace (protokol definovaný abstraktní třídou pro uzly `TreeNode`). Hlavní nevýhodou tohoto řešení je jeho nízká flexibilita a nutnost nešikovného zapouzdření prvků abstraktní třídou. Proto je využití šablon v takových případech pohodlnější a výhodnější především pro spravovatelnost zdrojového kódu a znovupoužitelnost (případně rozšiřitelnost).

Řešený příklad

Zadání: Vytvořte kostru pro třídu pro abstraktní datový typ binární strom s homogenním prvky s využitím OoJ se šablonami. Příklad je řešen v C++.

Řešení: T je v době psaní kódu neznámý typ; v době překladač však již známý je (případně až v několikátém průchodu zdrojovým kódem).

```
template<class T> class Node
{
public:
    Node(Node<T> *the_parent = NULL);
    T item;
    int number_of_items;
    Node<T> *parent;
    Node<T> *left_child;
    Node<T> *right_child;
};

template<class T> class BTree
{
public:
    BTree();
    ~BTree();
    void Insert(T item);
    ...
private:
    bool r, l;
    int node_count;
    Node<T> *root; // samotná datová struktura stromu
    void Insert(T item, Node<T> *tree, Node<T> *parent);
    void List_PreOrder(Node<T> *tree, Node<T> *parent);
    ...
};
```

5.3.9 Systémy s rolemi

motivace

Motivací k těmto systémům jsou případy, kdy objekt přetrvává v systému dlouhou dobu a postupně se vyvíjí a je potřeba tomu přizpůsobovat i schopnosti (protokol) tohoto objektu. Pro konkrétní použití tedy objekt může přebírat konkrétní roli. Objekt může během svého života v systému s rolemi vystupovat pod různými rolemi, které může postupně nabývat nebo pozbývat (za běhu systému, tj. za života objektu).

Definice!

Definice 5.3.3 *Objekt s více typy je objekt, který může mít v jeden okamžik více typů. Původ tohoto pojmu je v objektově orientovaných databázích, kde pak hovoříme o rolích objektu.*

Příklad 5.3.5 *Systém s rolemi - motivace.*

```
class Person is
  ...
endOfClass

class Student inherited from Person is
  ...
endOfClass

class Boarder inherited from Person is
  ...
endOfClass
```

V roce 1999 v systému s rolemi mohl vzniknout objekt Pavel, jež vystupuje pod rolí Student. O rok později si Pavel zaplatil stravování v menze a přibyla mu další role Strávník (angl. *Boarder*). Za další 4 roky Pavel ukončil studium a byla mu odebrána role Student, ale protože menza prodává obědy i nestudentům, tak mu role Strávníka zůstala.

S vícenásobnou dědičností bychom nevystačili ze dvou důvodů:

1. je možné velké množství kombinací, které by se velmi těžko udržovalo;
2. nelze předpovídat jaké další role bude nutné dotvořit v průběhu provozování samotného systému.

Systémy s rolemi používají většinou perzistentní objekty, což jsou použité objekty, které přežívají nezávisle na běhu nebo ukončení obsluhující aplikace.

Systémy s rolemi jsou většinou interpretované systémy, kde je potřeba provádět asociaci objektu a role za běhu (přidávání, odebrání rolí objektu). Například objektově orientované databáze nebo interpretované systém s dlouho-žijícími objekty.

Perzistentní objekty

Objekt či instanci označujeme za perzistentní, pokud přežívá rámec aplikace (čas kdy aplikace běží) a při dalším spuštění aplikace je tentýž objekt opět k dispozici v přesně stejném stavu jako měl při posledním ukončení aplikace.

Perzistence ale není jenom snímek objektové paměti (tzv. *snapshot*). Explicitní ukládání a načítání dat pro rekonstrukci objektů také nepovažujeme za perzistenci. Programátor by měl pouze deklarativně (např. pomocí klíčových slov) určit, které objekty se mají chovat jako perzistentní a které nikoliv (tzv. dynamické nebo krátkodobé).

Krátce si popíšeme dva možné přístupy k implementaci perzistentních objektů: implementace

- a) ukládat pouze stav objektu (hodnoty atributů) – metody objektu musí být dosažitelné jiným způsobem (například přes odkaz na třídu nebo role objektu, kde třídy/role jsou uloženy odděleně od objektů); z teoretického pohledu se však nejedná o plnohodnotnou perzistenci, jelikož lze pohodlně zajistit knihovnamí pro ukládání a načítání stavu objektů;
- b) ukládat stav objektu i jeho metody (případně třídu objektu) – protože na kompilovaných systémech je třeba v tomto případě řešit mnoho problémů (nutnost funkcí pro práci s kódem jako s daty, které bývají značně závislé na platformě), provádí se pouze překlad do mezikódu, jenž je následně interpretován virtuálním strojem.

5.4 Poznámky k implementaci OoJ

Při zamýšlení se nad implementací objektového jazyka je třeba brát v úvahu několik zásadních omezení nejčastěji používané von Neumannovy architektury počítačů, kdy nemáme k dispozici nic jako čistě objektovou paměť nebo mechanismus zasílání a směřování zpráv. Značné množství moderních OoJ řeší tyto nevýhody vytvořením tzv. *virtuálního stroje*, který vytváří pro jazyk interpretační nebo pseudopřekladačové prostředí obsahující objektovou paměť i snadnější práci se zprávami. Tomuto tématu se podrobněji věnuje sekce 5.4.2.

5.4.1 Manipulace se třídami

Pro ukládání atributů (datové části) objektu se v případě hybridního objektového přístupu jednoznačně nabízí využití datových struktur, jak je známe ze strukturovaných jazyků. Mírným problémem je u tohoto řešení nedostatečná variabilita takovéto struktury, která je pro ukládání některých objektů nezbytná.

Zásadní problémy, které je nutno řešit:

- ukládání metod
- dědičnost
- přístup k atributům v metodách
- instanční a třídní proměnné (atributy), instanční a třídní metody

Čistě objektové jazyky se těmito problémům vyhýbají díky běhu ve virtuálním stroji, o kterém se více dozvíte v sekci 5.4.2.

Problém uložení v paměti se týká především metod, které mají vždy různou délku (počet instrukcí po překladu). Do struktury, která v praxi reprezentuje objekt, se tak často vkládají pouze ukazatele na kód metod, který se nachází jinde v paměti.

Při podrobnějším prostudování následujícího příkladu zdrojového textu (př. 5.4.1) vidíme, že metoda `f1` je statická, a tudíž nemá přístup k instančním proměnným ani ostatním nestatickým metodám. Metody `f2` a `f3` jsou virtuální a implementace jejich uložení v paměti a následná invokace si bude žádat zvláštní pozornost, protože v době překladu nemůžeme zaručit, že se skutečně budou volat přímo tyto metody, či jejich redefinice v některém z potomků třídy `A`.

Příklad 5.4.1 Zdrojový text reprezentuje dvě třídy `A` a `B`. Na obrázku 5.1 pak vidíme grafické znázornění instance třídy `B` v klasické paměti.

```
class A
{
    public:
        int a, b;
        static int f1(int);
        float c;
        virtual float f2(int, float);
    private:
        int d;
        virtual int f3(int, int);
        float g(float);
}
```

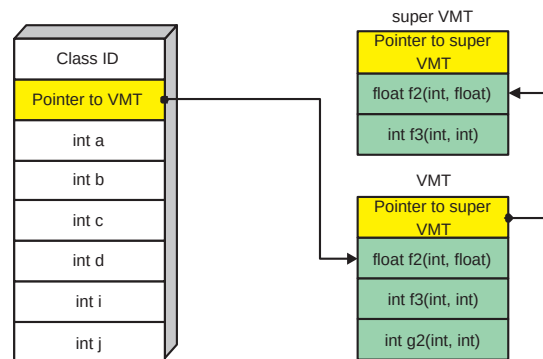
Problém s dědičností je patrný na následující třídě `B`, která dědí z předchozí třídy `A`. Její virtuální metoda `f2` je redefinována a v případě jejího volání s využitím polymorfismu musíme zajistit provedení kódu správné metody—té novější v případě objektu třídy `B`.

```
class B : A    // B dědí z třídy A
{
    public:
        int i;
        static int g1(int);
        float j;
        virtual float f2(int, float);
    private:
        virtual int g2(int, int);
}

B objB = new B();
objB.f2(0, 0.0) // příklad invokace f2 v objB
```

Na obrázku 5.1 vidíme nastíněno již plně vyhovující řešení, které splňuje všechny nastíněné požadavky:

- Statické metody a proměnné (neboli třídní metody a proměnné) nejsou uloženy v objektu ani VMT (u nich neexistuje v takovéto implementaci žádný polymorfismus).



Obrázek 5.1: Realizace uložení objektu třídy B pomocí tabulek virtuálních metod (VMT)

- Instance třídy B má dostupné jak svoje metody, tak metody všech předků (postupným procházením propojených VMT). Poznamenejme, že i VMT je implementována jako dynamická struktura a tedy obsahuje pouze reference na skutečný kód daných metod.

Úlohy k procvičení:

Zamyslete se nad možnostmi optimalizace tabulky virtuálních metod.

5.4.2 Virtuální stroj

Virtuální stroj je speciální softwarová vrstva, jejíž primárním účelem je odstínit pro běžící aplikaci hardwarová specifika počítače, na němž je vykonávána. Do toho zahrnujeme i proces samotného vykonávání kódu, díky čemuž je dosaženo naprosté nezávislosti na konkrétní platformě.

vykonávání kódu

Virtuální stroje se v přístupu k vykonávání kódu mohou značně lišit. V zásadě volí jeden z následujících přístupů

- přímá interpretace kódu ze syntaktického stromu zdrojového kódu (např. jazyk)
- kompilace do mezikódu a jeho následná přímá interpretace (Smalltalk-80)
- kompilace do mezikódu, jeho následný překlad to nativního strojového kódu a vykonání (SELF, Java)

bytekód

Jako mezikód se ve většině případů využívá tzv. *bytekód* (angl. *bytecode*), což je binární forma mezikódu s členěním po bytech. Tento

bytekód je nezávislý na platformě, oproti přímé interpretaci ze zdrojových kódů je rychlejší a nevyžaduje překladač jako součást virtuálního stroje. Většinou se jedná o rozšířený zásobníkový kód a tabulka jeho operačních kódů bývá silně optimalizována. Bytekód nebývá přímo závislý na jednom programovacím jazyce, ale obecně platí, že pro staticky typované programovací jazyky je komplikovanější, protože musí umět pracovat i s typovou informací.

Kvůli snaze zvýšit rychlost interpretace bytekódu se začal provádět jeho překlad do nativního strojového kódu konkrétní hostitelské platformy. Při tom se ve většině případů optimalizují pouze kritická místa, ve kterých program tráví nejvíce času. Díky tomu, že se tyto optimalizace provádí v čase běhu programu, bývají cílenější a účinnější než optimalizace prováděné při statickém překladu. Daní za často řádově vyšší rychlost, než kterou poskytuje interpretace bytekódu, je podstatně komplikovanější a hůře přenositelný virtuální stroj.

Zásadní roli hrají virtuální stroje při práci s pamětí. U objektově orientovaných jazyků se nechápe paměť jako sekvenční prostor pro ukládání dat, ale přímo jako množina objektů. Díky tomu většina moderních objektově orientovaných programovacích jazyků poskytuje automatickou správu paměti. Tu zajišťuje tzv. *garbage collector*² (GC), který automaticky vyhledává a ruší nepotřebné již neodkazované objekty. Implementace kvalitního GC je netriviální záležitost a většinou se v něm vhodně kombinuje několik implementačních strategií. Automatická správa paměti přirozeně vyžaduje jistou režii. Protože nad ní programátor nemá plnou kontrolu, může v krajních případech vést až k těžko odstranitelným výkonnostním problémům. Proto některé jazyky dovolují manuální a automatickou správu paměti kombinovat (např. ObjectiveC).

Některé jazyky (např. Smalltalk, SELF) dovolují používat tzv. *obrazy objektové paměti* (angl. *image, snapshot*). Při spuštění programu pak nedochází k postupné inicializaci objektů podle pokynů programu, ale je obnoven naposledy uložený stav. Tento přístup má celou řadu výhod (např. velice rychlý start aplikací) a pro objektové systémy je tato architektura paměti přirozená, nicméně v praxi se s ní zatím nelze setkat příliš často. Místo pouhého kódu programu je totiž nutné distribuovat celý obraz objektové paměti, který je objemnější. U obrazů se rovněž lze setkat s problematičtější modularitou aplikací.

Hlavní nevýhodou virtuálních strojů je podstatně vyšší režie, než v případě nativních programů. Rovněž větší odstup od prostředků hostitelského systému může být v některých případech velmi svazující. Nicméně tato architektura se díky svým nesporným výhodám

²Český ekvivalent termínu „garbage collector“ se zatím neustálil.

stále více prosazuje a to i případech, kdy výsledné programy nejsou vytvářeny s ohledem na použití na více hardwarových platformách.

5.4.3 Poznámka o návrhových vzorech

návrhový vzor

Návrhové vzory (angl. *design patterns*) jsou obecná znovupoužitelná řešení často se vyskytujícími problémů v programovém návrhu. Jedná se o popis postupu nebo šablonu, pomocí které pohodlně daný problém správně a efektivně vyřešíme. Objektově orientované návrhové vzory se typicky týkají vztahů a interakcí mezi třídami a objekty, aniž bychom přímo specifikovali konkrétní třídy a objekty naší konkrétní aplikace. Jedná se tedy o postupy s vysokou mírou abstrakce a z toho plynoucí flexibilitou a znovupoužitelností.

Definice!

Definice 5.4.1 *Návrhový vzor systematicky nazývá, vysvětluje a vyhodnocuje důležitý a v objektově orientovaných systémech se opakující návrh.*

Návrhové vzory lze zevrubně rozdělit takto:

1. Vytvářecí (instanciační, tvořivé) vzory — nepřímou cestou pro nás vytváří objekty, aniž bychom je museli vytvářet přímo, a poskytují nám tak větší flexibilitu programů (např. Jedináček, Tovární metoda, Abstraktní továrna, Prototyp)
2. Strukturální vzory — napomáhají při shlukování objektů do větších celků, jako například komplexní uživatelské rozhraní apod. (např. Dekorátor, Adaptér, Skladba, Kontejner, Proxy)
3. Vzory chování — pomáhají při definici komunikace mezi objekty v systému a toku řízení ve složitějších programech (např. Pozorovatel, Strategie, Šablonová metoda, Návštěvník)

V praxi asi nejběžnější návrhové vzory (podle [11]) jsou Abstraktní továrna, Adaptér, Dekorátor, Pozorovatel, Skladba, Strategie, Šablonová metoda a Tovární metoda.

Nyní si krátce popíšeme jeden ze základních jednoduchých návrhových vzorů z kategorie vytvářecích.

Jedináček

Jedináček (angl. *singleton*) je návrhový vzor, který omezuje možnosti vytvářet z třídy více jak jednu instanci. Existuje mnoho případů, kdy programátor potřebuje zajistit, že od dané třídy vznikne nejvýše jedna

instance, například přístupový bod k databázi nebo instance metatřídy (tedy třída). Jedináček bývá nejčastěji implementován pomocí veřejné statické metody, soukromé statické proměnné a zakázáním volání konstruktoru z ostatních tříd (nastavení viditelnosti konstruktoru na soukromou).

5.5 Zpracování - analýza, vyhodnocení, interpretace, překlad

Při zpracovávání objektově orientovaného jazyka máme tři možnosti:

1. Provést překlad zdrojového textu do samostatného modulu, který obsahuje přímo instrukce daného procesoru (případně také splňuje požadavky operačního systému). Výsledkem bývá tzv. nativní aplikace, podobná těm, které dostáváme jako výsledek překladu většiny strukturovaných a modulárních jazyků.
2. V případě čistých objektově orientovaných jazyků potřebujeme mít pro objektový program k dispozici paměť, která se chová jako objektová, a také instrukce pro zasílání zpráv. Tyto požadavky však nesplňuje dnešní von Neumannovská architektura počítačů, tak je nutno si vytvořit vrstvu mezi procesorem (případně operačním systémem) a naším objektovým programem. Takového mezivrstvě říkáme *virtuální stroj*, který je podrobněji popsán v sekci 5.4.2 na straně 78. V tomto případě pak překladač provádí transformaci zdrojového textu na kód, kterému rozumí onen virtuální stroj, jenž tento kód dokáže interpretovat.
3. Poslední možností je čistá interpretace zdrojového kódu bez překladu, která je však neefektivní (pomalá) a využívá se spíše výjimečně.

5.5.1 Překladač

U třídě založených OOJ se většinou praktikuje ukládání popisu každé třídy do zvláštního souboru³. Již díky tomuto přístupu je překladač OOJ minimálně tak složitý jako překladač modulárních jazyků, kdy

³ Výjimku ve způsobu uchovávání zdrojových textů tvoří některé OOJ využívající virtuální stroj, které mají vlastní grafické vývojové prostředí, které nám poskytuje různé nástroje od prohlížeče hierarchie tříd, až po návrh grafického uživatelského rozhraní vaší aplikace (např. SELF, Smalltalk). Samotné, do kódu virtuálního stroje přeložené, metody bývají v tomto případě součástí sdílené objektové paměti. Někdy se z praktických důvodů ukládá do formátovaného souboru i textová reprezentace tříd a metod (kvůli přenositelnosti definicí mezi různými

musí být schopen vytvořit graf závislosti pro překlad jednotlivých tříd a poté je ještě lineárně uspořádat, což se nemusí vždy podařit během jednoho průchodu. Další nezbytností je využívání lokálních tabulek symbolů během syntaktické analýzy, například pro atributy třídy. Náročným úkolem je i práce se jmennými prostory a různými modifikátory (např. modifikátor viditelnosti `internal` v C# pro určení viditelnosti dané entity pouze v rámci domovského jmenného prostoru).

Z hlediska lexikální a syntaktické analýzy OOJ nevyžadují žádné speciální přístupy nebo náročnější algoritmy než jazyky strukturované nebo modulární. Ovšem jak již víme, tak oba typy analýzy jsou velmi úzce spjatý s analýzou sémantickou, která je v OOJ o poznání složitější. Často je potřeba provádět v rámci sémantické analýzy i syntaktickou, takže je velmi relevantní efektivita lexikální a především syntaktické analýzy, kterou bychom měli v rozumné míře požadovat.

Sémantická analýza

Tato část překladu patří u OOJ mezi stěžejní. Uvádíme příklady sémantických kontrol a analýz, které však nemusejí být součástí každého objektového jazyka, z nichž některé dokonce mohou probíhat až za běhu programu a ne dříve. Situace se v tomto ohledu u staticky a dynamicky typovaných jazyků značně liší.

- Kontrola implicitního přetypování (časná/statická vazba)
- Kontrola explicitního přetypování objektu — kontrola typu (třídy) lze provádět až při samotném běhu programu, protože nejsme schopni při překladu s jistotou určit typ objektu, který se budeme pokoušet explicitně přetypovat (polymorfismus)
- Modifikátory viditelnosti

5.5.2 Interpret

V případě interpretace zdrojového kódu OOJ je situace lehce odlišná od klasických interpretů ve strukturovaných nebo modulárních jazycích. Většinou potřebujeme pro práci v interpretačním režimu k dispozici pracovní prostor (angl. *workplace*). Do tohoto prostoru jsou

obrazy objektové paměti). Čistě textový zápis se většinou omezuje na zápis zdrojových textů metod a zbylé programovací úkony lze více či méně provádět přes grafické rozhraní patřičného nástroje. Tato prostředí často obsahují i pokusy o implementaci vizuálního programování, kdy se vůbec nemusí psát kód.

ukládány všechny dočasné a potřebné objekty, aby se s nimi dalo pohodlně manipulovat. V interpretech OOJ se více než v jiných jazycích uplatňuje pravidlo, že doslova všechno je objekt, včetně pravdivostních i číselných hodnot. Interpretace probíhá průběžně, což způsobuje rychlou odezvu v případě testování a učení (podpora interaktivního inkrementálního programování, angl. *exploratory programming*). Samozřejmě v případě výpočtů náročných na výkon není interpretace zpravidla dobrým řešením i přes možnosti interní optimalizace, která se však navenek nesmí projevit změnou chování, ale jedine vylepšení rychlosti či spotřeby zdrojů.

Pojmy k zapamatování

Virtuální stroj, mezikód (bytekód), objektová paměť, tabulka virtuálních metod. Dále chápat úskalí implementace překladače či interpretu objektově orientovaného jazyka.

5.6 Závěr

Objektově orientované jazyky jsou velmi mocným nástrojem, který je vhodný především pro popis a manipulaci s komplexními strukturami a abstrakcemi. Je však potřeba být obezřetný vůči analytikům a manažérům, kteří mívají tendenci výhody objektové orientovanosti značně přeceňovat, což může ve výsledku vést až k nezdaru projektu. OOJ nabízí mnoho možností a postupů, ale tím také roste riziko, že se systém špatně navrhne. Přestože mají objektově orientované aplikace na dobré úrovni znovupoužitelnost, robustnost i udržitelnost, je nutné vybrat ten správný návrh, protože v případě chyby je přepsání celého systému někdy ještě náročnější než třeba u modulárních jazyků.

Kapitola poskytla čtenáři hodnotné informace ze zákulisí objektově orientovaných jazyků a o pokročilých vlastnostech spjatých s objektovou orientací. Pro usnadnění rozhodnutí, který jazyk je ten pravý pro odzkoušení té které vlastnosti uvádíme srovnávací tabulku 5.1 pěti významných zástupců objektově orientovaného programování.

Při hlubším studiu specifitějších vlastností OOJ je již efektivní se zaměřit na konkrétní vybraný jazyk, ke kterému bývá k dispozici odborná publikace nebo minimálně podrobná specifikace jazyka, včetně tipů a triků pro jeho implementaci.

Vlastosti / Jazyky	C++	Java	Python	Smalltalk	SELF
Typová kontrola	stat.	stat.	dyn.	dyn.	dyn.
Primitivní typy	ano	ano	ano	ne	ne
Orientace	třídní	třídní	třídní	třídní	obj.
Virtuální stroj	ne	ano	ano	ano	ano
Garbage collector	ne	ano	ano	ano	ano
Třídy	ano	ano	ano	ano	ne
Dědičnost	n	1	n	1	n
Reflexivita	ne	ano	ano	ano	ano
Kořenová třída	ne	ano	ne	ano	—
Šablony	ano	ne	ne	ne	ne
Abstraktní třídy	ano	ano	ne	ne	ne
Operátory	ano	ano	ano	b. z.	b. z.
Priorita operátorů	ano	ano	ano	ne	ne ⁴
Přetěžování	ano	ano	ano	ne	ne
Redefinice metod	ano	ano	ano	ano	ano
Konstruktory	ano	ano	ano	ne	ne
Destruktory	ano	ano	ano	ne	ne
Destruktor vždy volán	ano	ne	ano	—	—
Dynamické rozhraní objektů	ne	ne	ano	ne	ano
Vlastnosti	ano	ne	ano	ne	ne
Skrývání metod	ano	ano	ano	ne	ne
Dynamická dědičnost	ne	ne	ne	ne	ano
Obrazy objektové paměti	ne	ne	ne	ano	ano

Tabulka 5.1: Srovnání vlastností vybraných objektově orientovaných jazyků (zkratky: *stat.* = statická, *dyn.* = dynamická; *obj.* = objektová; 1 = jednoduchá, *n* = násobná; *b. z.* = binární zprávy)

Úlohy k procvičení:

Vyberte si objektově orientovaný jazyk, který není v tabulce 5.1 a do tabulky pro něj doplňte nový sloupec. Pro inspiraci uvádíme několik vhodných kandidátů: PHP 5, JavaScript, C#, VisualBasic.NET, ObjectiveC, CLOS, Object Pascal (Delphi) a Ruby.

⁴vynucené závorkování

Závěr

V této kapitole jsme se zaměřili na méně sourodý výčet a popis především zajímavých a moderních vlastností OOP, které jsou většinou vůči tomuto paradigmatu ortogonální (uplatnitelné i v jiných paradigmatech).

Po dokončení podrobnějšího dělení jazyků podle způsobů typové kontroly následují podkapitoly probírající témata týkající se nejen třídnicích OOJ jako řízení toku v hybridních OOJ, modifikátory viditelnosti a přetěžování metod. Mezi ještě obecnější mechanismy patří bezesporu výjimky a šablony. Zmínka je i o perzistentních objektech a systémech s rolemi.

Zbytek kapitoly se věnuje úskalím překladu a interpretace objektově orientovaných jazyků.

Další zdroje

Kvalitní publikace pro studium pokročilejších vlastností a mechanismů implementace OOJ již nejsou tak snadno dostupné, v českých překladech zpravidla jen pro nejrozšířenější jazyky (C++, Java):

- Arnold, K., Gosling, J., Holmes, D.: The Java™ Programming Language, Fourth Edition, Addison Wesley, 2005, ISBN 0-321-34980-6.
- Goldberg, A., Robson, D.: Smalltalk-80 - The Language and Its Implementation (Blue Book), Addison Wesley, 1983.
- Eckel B.: Thinking in C++ (Volume 1 & Volume 2), Prentice Hall, 2000-2003.

Dostupné na URL <http://mindview.net/Books/TICPP/ThinkingInCPP2e.html> (únor 2006).

Velmi přínosnou knihou pro budoucí dobré návrháře a programátory je popis návrhových vzorů:

- Gamma, E., Helm, R., Johnson, R., Vlissides, J.: Design Patterns – Elements of Reusable Object-Oriented Software, Addison Wesley, 1995.

Kapitola 6

Závěr

Tento díl celé publikace představuje v několika kapitolách objektově orientované metody analýzy, návrhu a implementace, kam spadá především objektově orientované programování.

Hlavní úsilí je kladeno na vysvětlení objektově orientovaného paradigmatu a doplnění pasáží, které nejsou přímo dostupné v české literatuře. Text se nezaměřuje na konkrétní jazyk a jeho syntaxi a sémantiku, ale snaží se poskytnout studentovi ucelený výklad základních i pokročilejších vlastností jak z hlediska užívání OOJ, tak z hlediska zpracovávání a rozšiřování existujících implementací.

Po absolvování by student měl být schopen podle popsaných vlastností objektově orientovaných jazyků tyto jazyky nejen rozpoznat, ale i aktivně studovat jejich pokročilejší vlastnosti, a případně se rozhodovat o použitelnosti toho či onoho jazyka pro řešení konkrétního problému.

Pro doplnění znalostí jsou vhodné moduly pojící se s výkladem některého konkrétního programovacího jazyka (např. C++, Java nebo Smalltalk).

Obsah modulu obsahuje pouze nejn nutnější informace a nástin problematiky. Pro úplné zvládnutí problematiky je vhodné rozšířit si vědomosti nějakou vhodnou doporučenou literaturou jak z oblasti objektově orientovaných jazyků, tak třeba z okrajových témat.

Rejstřík

- Abstrakce, 11, 13, 16
- Abstraktní datový typ, 18
- Ada, 62
- Adresát, viz Příjemce
- Agregace, 52
 - silná, viz Kompozice
- Alokace
 - na haldě, 19
 - na zásobníku, 19
- Analýza
 - lexikální, 79
 - sémantická, 79
 - syntaktická, 79
- Argument, viz Parametr
- Asociace, 51
 - vícenásobná, 51
- Atribut, 12, 37, 48
 - modifikace, 37
 - odvozený, 48
 - výběr, 37
- base, 26
- Binární strom, 70, 71
- Black Box, viz Černá skříňka
- Booch, Grady, 11, 46
- Bytecode, viz Byte kód
- Byte kód, 67, 75
- C, 11, 23, 62
- C++, 11, 14–16, 19, 23, 25, 62, 64, 71, 81
- C#, 16, 19, 21, 25, 64, 66, 79
- catch-blok, 69
- Chování, 11
- CLOS, 64
- Černá skříňka, 13
- Dahl, Ole-Johan, 11
- Datový typ, 60
 - parametrizace, 70
- Dědičnost, 11, 14, 17, 21
 - dynamická, 30
 - implementace, 21
 - jednoduchá, 21
 - přímá, 22
 - rozhraní, 22
 - vícenásobná, 19, 21, 30, 64
 - problémy, 65
 - vyžadovaná, 23
- Deep copy, viz Hluboká kopie
- Delegace, 29
- Delegovaný objekt, 29
- Delphi, 23, 25
- Design Pattern, viz Návrhový vzor
- Destruktor, 20
 - virtuální, 20
- Diagram, 46
 - datových toků, 54
 - komponent, 54
 - objektů, 53
 - případů použití, 54
 - rozmístění zdrojů, 54
 - sekvence, 55
 - seskupení, 54
 - spolupráce, 55
 - stavový, 56
 - tříd, 50
- Diskriminátor dědičnosti, 52
- Exception, viz Výjimky
- Finalizační sekce, 68

- Finalizace, 20
- Formální návrh, 42
- Fortran, 62
- friend, 64
- Garbage Collector, 28, 76
- Generalizace, 51
- get, 68
- Goldbergová, Adele, 11
- Gosling, James, 11
- Grafické uživatelské rozhraní, 11
- Haskell, 66, 69
- Hierarchie tříd, 21, 22
- Hluboká kopie, 19
- Identita, 15
- Image, viz Objektová paměť
- Implicitní předávání odkazem, 20
- Inicializace, 19
- Instance, 18, 77
- Instanciace, 18
- Instanční metoda, 22
- Instanční proměnná, 22
- Interface, viz Rozhraní
- internal, 64, 79
- Interpret, 79
- Invariant, 13
- Invokace metody, 37
- Jacobson, Ivar, 11, 46
- Java, 11, 16, 19, 21, 23, 25, 62, 66, 75, 81
- JavaScript, 30
- Jazyky
 - beztypové, 61
 - netypované, 16, 36, 61
 - objektové
 - čistě, 25
 - hybridně, 25
 - typované, 61
 - dynamicky, 16, 25, 27, 30, 62
 - silně, 62
 - staticky, 14, 16, 25, 27, 61
 - založené na prototypch, 28
 - založené na třídách, 17
- Jedináček, 77
- Jmenný prostor, 63
- Kay, Alan, 10, 11
- Klasifikace jazyků, 60
- Klíčové slovo, 19
- Klonování, 28
- Kompozice, 52
- Koncepty, 12
- Konstruktor, 19
- Kvalifikace jména, 26
- λ -kalkul, 61
- Lisp, 11
- Literál, 38
 - vytvoření objektu, 37
- Mělká kopie, 19
- Memory leaks, viz Úbytky paměti
- Metamodel, 56
- Metatřída, 22
- Metoda, 12
 - instanční, viz Instanční metoda
 - odvozená, 68
 - statická, 74
 - třídní, viz Třídní metoda
 - virtuální, 27
- Mezikód, 75
- Míra abstrakce, 13
- Mnohotvárnost, viz Polymorfismus
- Model
 - chování, 47
 - reálný, 10
 - softwarový, 10
 - struktury, 47
- Modifikátor viditelnosti, 14, 49, 63

- Modifikace metody, 37
Modula-3, 20, 62
Nadtyp, 23
Namespace, viz Jmenný prostor
Násobnost vztahu, 51
Nativní aplikace, 78
Návrhový vzor, 77
 chování, 77
 strukturální, 77
 vytváření, 77
Nedefinovaná hodnota, 15
new, 19
Nultý parametr, 26
Nygaard, Kristen, 11

Oberon, 19
Object, 25
Object Pascal, 23, 25
ObjectiveC, 76
Objekt, 11, 13, 37, 48
 perzistentní, 72
 reálný, 11
Objektová paměť, 72, 76
Odesílatel, 12
Operace, 12, 48
 abstraktní, 49, 70
Optimalizace, 25, 76
Ortogonalita, 60

Parametr, 14
Pascal, 62
Perzistence, 72
PHP5, 19
Podtyp, 22, 23
 skutečný, 23
Pohled, 46
Polymorfismus, 14, 26
 generický, 67
Potomek, 21
Pracovní prostor, 79
private, viz Modifikátor viditelnosti
Programování
 interaktivní inkrementální, 80
 objektově orientované, 10
 objektově založené, 49
 vizuální, 79
Proměnná, 37
 instanční, viz Instanční proměnná
 třídní, viz Třídní proměnná
 vázaná, 38
 volná, 38
protected, viz Modifikátor viditelnosti
Protokol objektu, 12
Prototyp, 30
Prvek, 47
Překladač, 78
Přetěžování metod, 64
Přetěžování operátorů, 64
Příjemce, 12
Přiřazení, 15
Pseudoproměnná, 26
public, viz Modifikátor viditelnosti
Python, 19, 64, 81

Realizace, 51
Redefinice metod, 25
Redukce, 39
 invokace, 39
 modifikace, 39
Reference, 15, 20
rise, 69
Robustnost, 14, 16
Rodič, 21
Role, 71
Rozhraní, 66
Rozhraní objektu, viz Protokol objektu
Rozšiřitelnost, 14, 16
Ruby, 75
Rumbaugh, Jim, 11, 46
Rušení instancí, 20

- Rušení objektů, 28
- Rys, 29
- Sdílení
 - chování, 14
 - kódu, 14
- SELF, 30, 31, 39, 62, 75, 76, 78, 81
- self, 26, 37
- Sémantická akce, 37
- Sémantická kontrola, 79
- set, 68
- Shallow copy, viz Mělká kopie
- Shluky objektů, 29
- Shoda, 15
- ς-kalkul, 36, 61
 - sémantika, 39
 - syntaxe, 37
- ς-term, 37
- Simula, 11, 20
- Singleton, viz Jedináček
- Slot, 28, 39
 - rodičovský, 29
- Smalltalk, 11, 16, 19–22, 25, 62, 64, 75, 76, 78, 81
- Směrování zpráv, 16, 27
- Snapshot, viz Objektová paměť
- Specializace, viz Generalizace
- static, 23
- Statická metoda, 23
- Statická proměnná, 23
- Stav, 12
- Stereotyp, 49
 - rozhraní, 49
- Stroustrup, Bjorn, 11
- Subclass, viz Podtyp
- Substituce, 38
- super, 26
- Superclass, viz Nadtyp
- Systém s rolemi, 72
- Šablona, 69, 70
- Šablonový typ, 70
- Tabulka virtuálních metod, 27, 68
- this, 26, 37
- throw, 69
- TObject, 25
- trait, viz Rys
- try, 69
- Třída, 17, 18, 48
 - abstraktní, 49
 - asociační, 51
 - parametrizovaná, 50
- Třídní metoda, 22
- Třídní proměnná, 22
- Typ, 23
- Typová chyba, 61
- Typová inference, 61
- Typová kontrola, 14
 - dynamická, 62
 - silná, 61
 - statická, 62
- Typový systém, 60
- Úbytky paměti, 20
- Udržovatelnost, 14, 16
- Ukazatel, 15
- UML, 11, 41, 46
- Unified Modeling Language, viz UML
- union, 62
- Variant, 13
- Vazba
 - časná, 27
 - pozdní, 27
 - přes jméno, 67
- Viditelnost, viz Modifikátor viditelnosti
- View, viz Pohled
- Virtuální stroj, 73, 75, 78
- VMT, viz Tabulka virtuálních metod
- von Neumann, John, 17, 73
- Výjimky, 68
- Vztah, 47

Workplace, viz Pracovní prostor

Zapouzdření, 13, 16

Zasílání zpráv, 11, 12, 15

Závislost, 53

Znovupoužitelnost, 14, 17

Zpráva, 55

Literatura

- [1] Abadi, M., Cardelli, L.: *A Theory of Objects*, Springer, New York, 1996, ISBN 0-387-94775-2.
- [2] Arlow, J., Neustadt, I.: *UML a unifikovaný proces vývoje aplikací*, Computer Press, Brno, 2003 (překlad do češtiny, Addison-Wesley, 2002), ISBN 80-7226-947-X.
- [3] Arnold K., Gosling J., Holmes D.: *The JavaTM Programming Language*, Fourth Edition, Addison Wesley, 2005, ISBN 0-321-34980-6.
- [4] Brender, R.F.: *The BLISS programming language: a history*, Softw. Pract. Exper. **32**, 2002, pp. 955–981.
- [5] Brodský, J., Staudek, J., Pokorný, J.: *Operační a databázové systémy*, Technical University of Brno, 1992.
- [6] Cattell, G. G.: *The Object Database Standard ODMG-93*, Release 1.1, Morgan Kaufmann Publishers, 1994.
- [7] Cooper, J. W.: *C# Design Patterns - A Tutorial*, Addison-Wesley, 2003, ISBN 0-201-84453-2.
- [8] Eckel, B.: *Thinking in Java*, 3rd Edition, Prentice-Hall, 2002.
- [9] Eckel, B.: *Thinking in C++*, 2nd Edition, Volume 1 & Volume 2, Prentice-Hall, 2000-2003.
- [10] Ellis, M. A., Stroustrup, B.: *The Annotated C++ Reference Manual*, AT&T Bell Laboratories, 1990, ISBN 0-201-51459-1.
- [11] Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns – Elements of Reusable Object-Oriented Software*, Addison Wesley, 1995.
- [12] Goldberg, A., Robson, D.: *Smalltalk-80 – The Language and Its Implementation* (Blue Book), Addison Wesley, 1983.

-
- [13] Finkel, R. A.: *Advanced Programming Language Design*, Addison-Wesley, California, 1996.
 - [14] Gordon, M. J. C.: *Programming Language Theory and its Implementation*, Prentice Hall, 1988, ISBN 0-13-730417-X, ISBN 0-13-730409-9.
 - [15] Gray, P. M. D., Kulkarni, K. G., Paton, N. W.: *Object-Oriented Databases*, Prentice Hall, 1992.
 - [16] Chonoles M. J., Schardt, J. A.: *UML 2 for Dummies*, 2003, ISBN 0-764-52614-6.
 - [17] Jones, M. P.: *Fundamentals of Object-Oriented Design in UML*, Addison-Wesley, 2000.
 - [18] Kay, A. C.: *The Early History of Smalltalk*, ACM, 1993.
 - [19] Khoshafian, S., Abnous, R.: *Object Orientation. Concepts, Languages, Databases, User Interfaces*, John Wiley & Sons, 1990, ISBN 0-471-51802-6.
 - [20] Křivánek, P.: *Podpora beztrždního programování ve Squeak Smalltalku*, diplomová práce, FIT VUT Brno, 2005.
 - [21] Křivánek, P.: *Squeak Smalltalk*, seriál článků, Root.cz, ISSN 1212-8309, 2003-2005.
 - [22] Laštovička, J.: *Hranice prototypových programovacích jazyků*, diplomová práce, PřF UP Olomouc, 2011.
 - [23] Leroy, X.: *The Objective Caml system, documentation and user's guide*, 1997, Institut National de Recherche en Informatique et Automatique, Francie, Release 1.05, <http://pauillac.inria.fr/ocaml/htmlman/>.
 - [24] Lindholm, T., Yellin, F.: *The Java Virtual Machine Specification*, Addison-Wesley, 1996, ISBN 0-201-63452-X.
 - [25] Merunka, V.: *Objektové metody a přístupy - Smalltalk-80*, učební text.
 - [26] Prata, S.: *Mistrovství v C++*, Computer Press, 2004, ISBN: 80-251-0098-7.
 - [27] Odersky, M., Wadler, P.: *Pizza into Java: Translating theory into practice*, Proc. 24th ACM Symposium on Principles of Programming Languages, January 1997.

-
- [28] Object Management Group (OMG): *Unified Modeling Language - Specification*, Version 1.2, 1998.
 - [29] Richards, M.: *The BCPL Reference Manual*, Memorandum M-352, Project MAC, Cambridge, MA, USA, 1967.
 - [30] Robinson, S. a kolektiv: *C# Professional*, John Wiley and Sons, 2003.
 - [31] Sebesta, R. W.: *Concepts of Programming Languages*, Forth Edition, Addison-Wesley, 1999, ISBN 0-201-38596-1.
 - [32] Schmidt, D. A.: *The Structure of Typed Programming Languages*, MIT Press, 1994.
 - [33] Schmuller, J.: *Sams Teach Yourself UML in 24 Hours*, Third Edition, 2004, ISBN 0-672-32640-X.
 - [34] Švec, M.: *Programovací jazyk a aplikační prostředí SELF*, FIT VUT Brno, 2004.